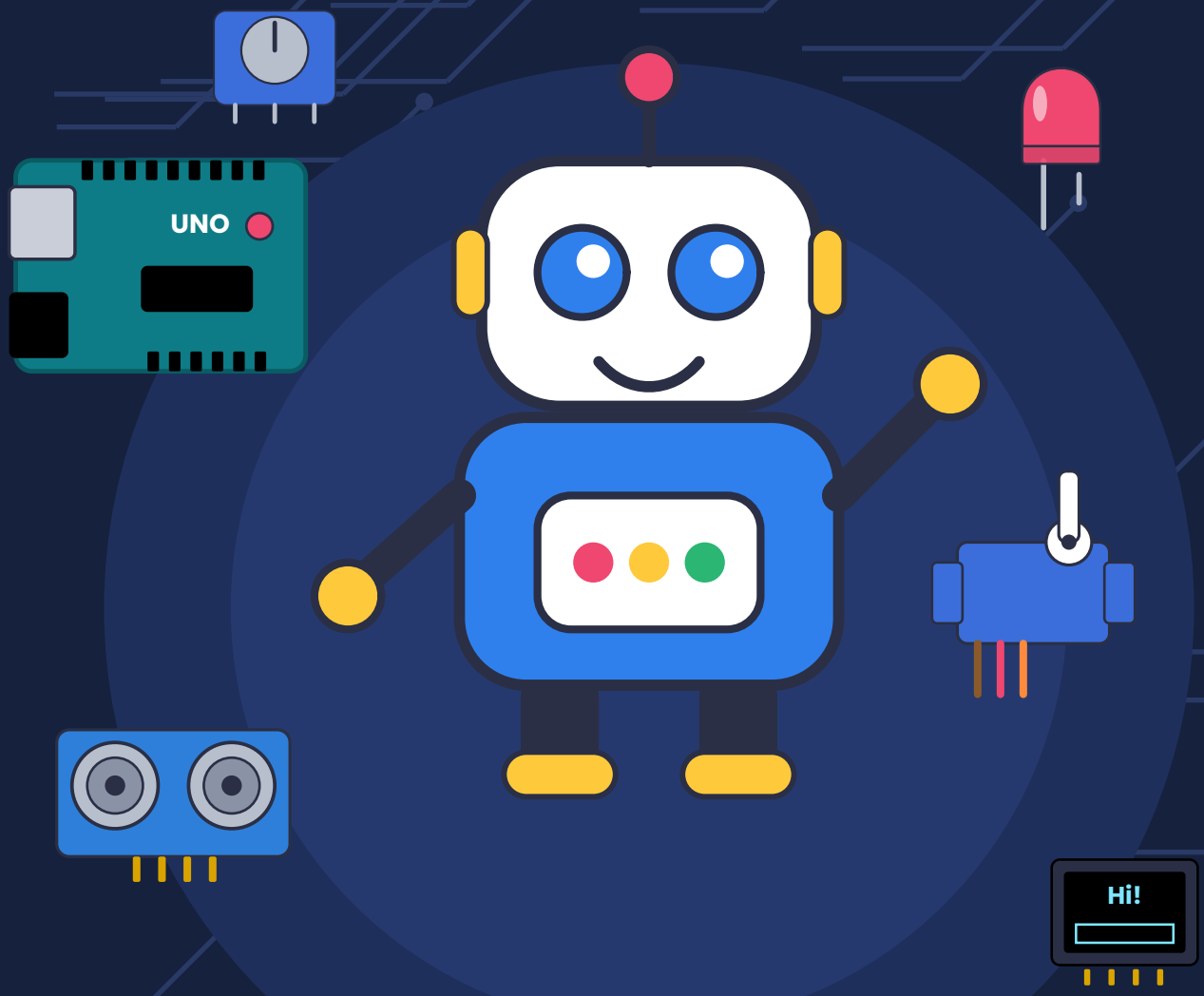
A HANDS-ON BOOKLET FOR YOUNG MAKERS

Little Makers

Learn Embedded Systems and Robotics with Arduino



1 Meet the Tools tools + software	**2** Basic 9 projects	**3** Intermediate 9 projects	**4** Advanced 7 projects

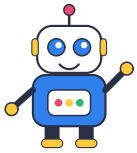
25 Projects | Hardware + Software | Grade 5 and above

Simple words. Big pictures. Step-by-step. No experience needed.

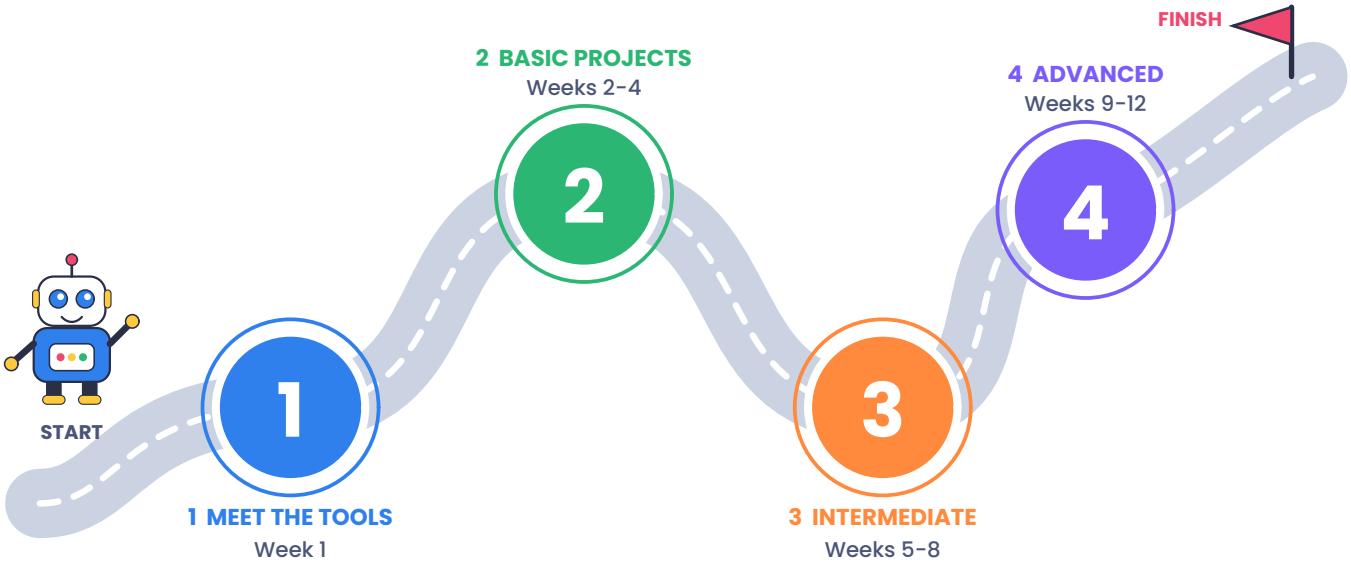
Designed by Engr. Abdul Basit

START HERE

Your Learning Map



Welcome, young maker! This map shows your journey. You start with the tools, then build small projects, then bigger ones, and at the end you build real robots. Go step by step. Each week has about 2 sessions of 60 minutes.



12-WEEK LEARNING SCHEDULE

Week	What you learn	Where	Time
1	Meet the tools: parts, breadboard, Arduino IDE	Section 1	2 sessions
2	Hello World, LED Blink, Traffic Light	Projects 1 - 3	2 sessions
3	Button, Buzzer Melody, Knob Reader	Projects 4 - 6	2 sessions
4	Fading LED, Night Light, RGB Mixer	Projects 7 - 9	2 sessions
5	LCD and OLED screens	Projects 10 - 11	2 sessions
6	Servo motor and distance sensor	Projects 12 - 14	2 sessions
7	Temperature sensor and motion alarm	Projects 15 - 16	2 sessions
8	Plant monitor and parking helper	Projects 17 - 18	2 sessions
9	Motor driver, speed and direction	Projects 19 - 20	2 sessions
10	Line follower robot	Project 21	2 sessions
11	Obstacle avoider and Bluetooth car	Projects 22 - 23	3 sessions
12	Weather station and secret door lock	Projects 24 - 25	3 sessions

Tip for teachers and parents: A session can be a school lesson or a home study hour. Finish the challenge at the end of each project before you move on.

INSIDE THIS BOOKLET

Table of Contents

SECTION 1 Meet the Tools 4

1.1 What is an embedded system? 5
1.2 Meet the Arduino UNO 6
1.3 The parts in your kit 7
1.4 Breadboard, LED and resistor 8
1.5 Electricity in simple words 9
1.6 The software: Arduino IDE 10
1.7 How a sketch works 11
1.8 Safety and good habits 12

SECTION 2 Basic Projects 13

01 Hello World! 14
02 LED Blink 16
03 Traffic Light 18
04 Button Controls LED 20
05 Buzzer Melody 22
06 Knob Reader 24
07 Fading LED 26
08 Night Light (LDR) 28
09 RGB Colour Mixer 30

SECTION 3 Intermediate Projects 32

10 LCD Message Board 33
11 OLED Display 35
12 Servo Sweep 37
13 Servo Knob Control 39
14 Distance Meter 41
15 Weather on the LCD 43
16 Motion Alarm 45
17 Plant Water Monitor 47
18 Smart Parking Helper 50

SECTION 4 Advanced Projects 53

19 Motor Driver Basics 54
20 Speed and Direction Control 56
21 Line Follower Robot 58
22 Obstacle-Avoiding Robot 61
23 Bluetooth Remote Car 64
24 Mini Weather Station 67
25 Secret Door Lock 70

APPENDIX 73

A Pin cheat sheet 73
B When something goes wrong 74
C Words to know 75
D My project tracker 76
E Maker certificate 77

HOW EVERY PROJECT PAGE WORKS

1 **What is it?**
Read the idea and the objective.

2 **Hardware**
Collect the parts from your kit.

3 **Connect**
Follow the colour wires in the diagram.

4 **Code and try**
Type the code, upload and test.

SECTION 1 | LEVEL: WARM-UP

Meet the Tools

Get to know the hardware and the software.



Before we build, we need to meet our helpers. In this section you will learn what an embedded system is, which parts are in your kit, and how to use the Arduino IDE on your computer. Take your time. Every maker starts here.

IN THIS SECTION

1.1	What is an embedded system?	1.2	Meet the Arduino UNO
1.3	The parts in your kit	1.4	Breadboard, LED and resistor
1.5	Electricity in simple words	1.6	The software: Arduino IDE
1.7	How a sketch works	1.8	Safety and good habits

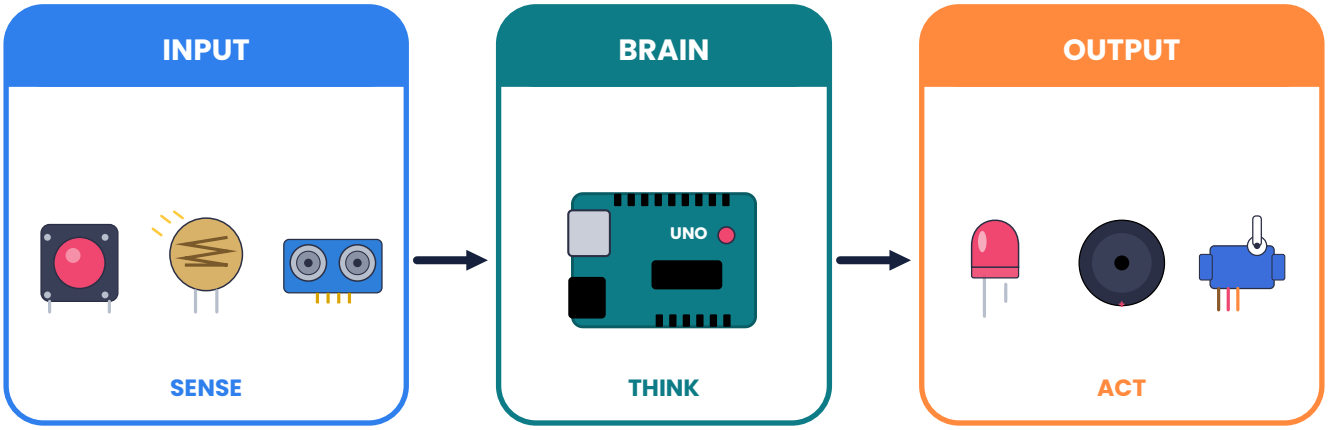
ROBO SAYS
You do not need to remember everything on these pages. Come back and look at them when you need help. Pictures are your friends!

WHAT YOU WILL HAVE AT THE END

- You know the main parts of the Arduino UNO.
- You can use a breadboard and read a resistor.
- You have the Arduino IDE ready on your computer.
- You understand setup() and loop(), the heart of every program.

1.1 What is an Embedded System?

An embedded system is a tiny computer hidden inside a machine. It has one small job and it does it all day.



Every embedded system works in the same three steps: **sense** (read the world), **think** (decide), **act** (do something). A robot is an embedded system that can also move.

A PERSON HAS	A PERSON HAS	A PERSON HAS
Eyes, ears, skin	**Brain**	**Hands, legs, voice**
A ROBOT HAS	A ROBOT HAS	A ROBOT HAS
Sensors	**Microcontroller**	**Motors, LEDs, speakers**

EMBEDDED SYSTEMS AROUND YOU

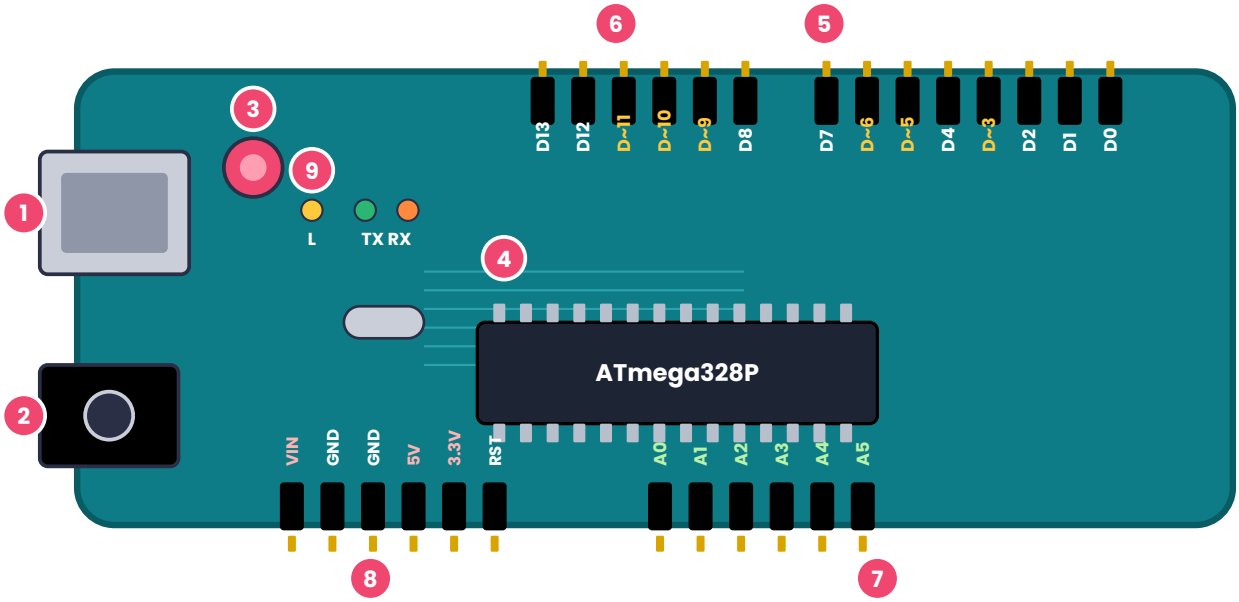
Machine	How it works
Microwave oven	Button (input) - timer in the chip - heater and light (output)
Traffic light	Timer in the chip - changes the red, yellow and green lights
Automatic door	Motion sensor sees you - the chip decides - motor opens the door
Washing machine	Sensors for water level - chip picks the program - motor spins the drum
Smart watch	Sensors count steps and heartbeat - chip counts - screen shows the numbers

A **WORDS TO KNOW**
Embedded system = a small computer inside a machine. **Microcontroller** = the tiny brain chip. **Sensor** = a part that reads the world. **Actuator** = a part that moves or lights up.

1.2 Meet the Arduino UNO

The Arduino UNO is a small board with a brain chip. You will use it in all 25 projects.

ARDUINO UNO (seen from above)



1	**USB port**	Gives power and lets you upload code from the computer.
2	**Power jack**	Use a 7-12 V adapter or a battery here when the USB is not connected.
3	**Reset button**	Press it to start your program again from the beginning.
4	**Microcontroller**	The tiny brain chip (ATmega328P). It runs your program.
5	**Digital pins 0-13**	Each pin can be an input or an output. They read or send ON / OFF.
6	**PWM pins (~)**	Pins 3, 5, 6, 9, 10, 11 can give smooth power levels, not only ON / OFF.
7	**Analog pins A0-A5**	These pins read sensors that give smooth values (0 to 1023).
8	**Power pins**	5V, 3.3V and GND (ground) give power to your parts.
9	**Built-in LED (L)**	A small LED connected to pin 13. It blinks in the first test.

FUN FACT
Pin means a small metal hole or leg where you connect a wire. Digital pins are named D0 to D13. Analog pins are named A0 to A5. Pins with a ~ sign can make smooth (PWM) power.

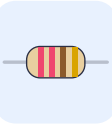
1.3 The Parts in Your Kit

Here are all 20 parts you will use. Look at the pictures. Each part has one special job.



1

LED
A tiny light. It needs a resistor.



2

Resistor
Limits the current to protect parts.



3

Push button
Gives a signal when you press it.



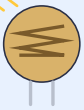
4

Buzzer
Makes beeps and musical notes.



5

Potentiometer
A knob that gives smooth values.



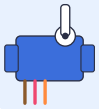
6

LDR
A sensor for light and dark.



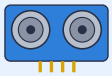
7

RGB LED
Red, green and blue light in one.



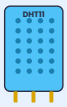
8

Servo motor
Turns to an exact angle.



9

Ultrasonic sensor
Measures distance with sound.



10

DHT11
Measures temperature and humidity.



11

PIR sensor
Feels moving people (heat).



12

Soil sensor
Checks if soil is wet or dry.



13

LCD 16x2
Shows 2 lines of text.



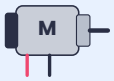
14

OLED screen
Small, sharp screen for text and shapes.



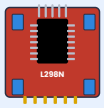
15

IR sensor
Sees black lines and close objects.



16

DC motor
Spins wheels. Needs a driver.



17

Motor driver
Powers motors for the Arduino.



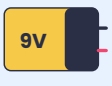
18

Bluetooth HC-05
Talks to a phone without wires.



19

Keypad 4x4
16 buttons for numbers and PINs.



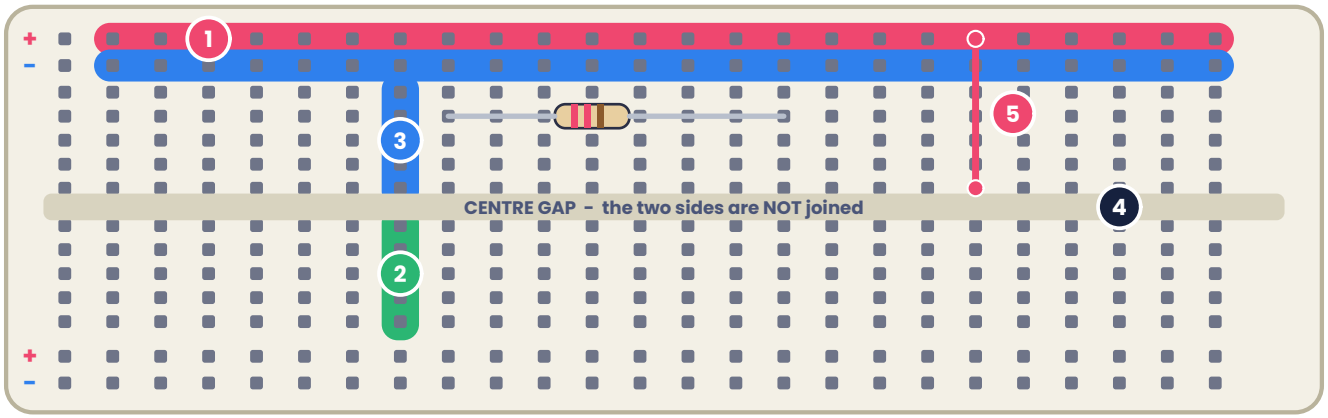
20

Battery
Power for motors and robots.

ROBO SAYS
You do not need all parts at once. Each project tells you which parts to take from the box.

1.4 Breadboard, LED and Resistor

A breadboard lets you build a circuit without glue or hot solder. You just push the legs in.



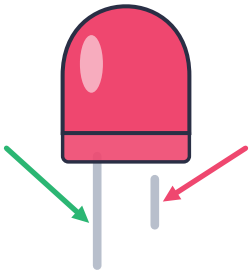
1	**Power rails**	(red + and blue -). All holes in one long row are joined. Use them for 5V and GND.
2	**Green strip**	The 5 holes in one short column are joined together. Put one leg in each hole.
3	**Blue strip**	This is a different group. The centre gap keeps the two sides apart.
4	**Centre gap**	Chips and buttons sit across it so their legs do not touch.
5	**Jumper wire**	It makes a bridge from one place to another. Red is usually power, black is ground.

THE LED HAS A FRONT AND A BACK



LED symbol

Longer leg = POSITIVE (+)
goes to the resistor and the pin



Shorter leg = NEGATIVE (-)
goes to GND. Also the flat edge.

HOW TO READ A RESISTOR

A resistor has coloured bands. They are a secret number code. You need only these four:



220 ohm
red - red - brown



1 k ohm
brown - black - red



2 k ohm
red - black - red

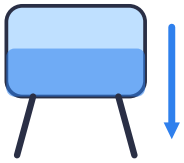


10 k ohm
brown - black - orange

1.5 Electricity in Simple Words

You cannot see electricity, but you can imagine it like water in pipes.

VOLTAGE (V)



The PUSH

Like the height of a water tank. More height = more push.

CURRENT (A)



The FLOW

Like how much water moves in the pipe every second.

RESISTANCE (ohm)



The SQUEEZE

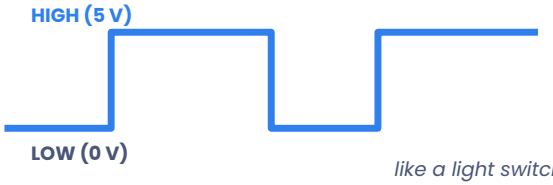
Like a narrow pipe. It makes the flow smaller.

A **circuit** is a loop. Power leaves the **5V** pin, goes through the parts and comes back to **GND** (ground). If the loop is broken, nothing works. That is why a wire that is not pushed in all the way can stop a project.

> **CAREFUL!**
> **Why does an LED need a resistor?** An LED lets too much current go through. It becomes very hot and breaks. The resistor squeezes the flow, so the LED stays safe.

DIGITAL AND ANALOG SIGNALS

DIGITAL = ON or OFF



like a light switch

ANALOG = ANY VALUE



0 to 1023 (0 V to 5 V)

like a dimmer or a knob

1.6 The Software: Arduino IDE

IDE means Integrated Development Environment. It is the program where you write code and send it to your board.

GET IT READY (ONE TIME ONLY)

1. Ask an adult to open **arduino.cc/en/software** and download the **Arduino IDE** for your computer (Windows, Mac or Linux). It is free.
2. Install it. Click Next and Yes when the computer asks.
3. Connect the Arduino UNO to the computer with the USB cable. A green light turns on.
4. Open the Arduino IDE. Choose **Tools > Board > Arduino AVR Boards > Arduino Uno**.
5. Choose **Tools > Port** and pick the port with the name of your Arduino.

A TOUR OF THE WINDOW



1	**Verify**	Checks your code for mistakes. Nothing is sent to the board.
2	**Upload**	Sends your code to the Arduino. The board then starts to run it.
3	**Board and port**	Tell the IDE which board you use and which USB port it is on.
4	**Serial Monitor**	Opens a window to read messages from the Arduino.
5	**Editor**	This is where you type your code. It is called a sketch.
6	**Output window**	Shows messages and errors in red. Read them. They help!

First test: choose **File > Examples > 01.Basics > Blink**. Click **Upload**. The small orange L light on the board should blink. Your setup works!

1.7 How a Sketch Works

A program for Arduino is called a **sketch**. It always has two main parts.



THE 8 MOST IMPORTANT COMMANDS

`pinMode(pin, OUTPUT)`	Say if a pin sends (OUTPUT) or listens (INPUT).
`digitalWrite(pin, HIGH)`	Turn a pin ON (HIGH) or OFF (LOW).
`digitalRead(pin)`	Check if a pin is HIGH or LOW. Great for buttons.
`analogRead(pin)`	Read a sensor value from 0 to 1023.
`analogWrite(pin, 0-255)`	Give smooth power (PWM) to LEDs and motors.
`delay(ms)`	Wait. 1000 milliseconds = 1 second.
`Serial.println("Hi")`	Send a message to the Serial Monitor.
`if (...) { } else { }`	Make a decision.

CODE RULES (THE GRAMMAR OF ARDUINO)

- End each command with a **semicolon ;**
- Every opening brace **{** needs a closing brace **}**.
- Capital letters matter. **delay** is right. **Delay** is wrong.
- **// text** is a comment. The Arduino ignores it. Use comments to explain your code.
- A variable is a box with a name. **int count = 5;** stores the number 5 in a box called count.

```
1  int count = 0; // make a box called count
2
3  void setup() {
4    Serial.begin(9600); // runs once
5  }
6
7  void loop() {
8    count = count + 1; // add 1 every time
9    Serial.println(count); // show the number
10   delay(1000);
11 }
```

1.8 Safety and Good Habits

Makers are careful. Follow these rules in every project.

Power OFF first
Unplug the USB cable (and battery) before you change any wire.

Check twice
Look at the diagram again before you connect the power.

Never touch hot parts
Motor drivers and resistors can become hot. Wait to let them cool.

No water near the board
Water and electronics are not friends. Only the soil probe goes in the plant pot.

Use the right power
USB gives 5 V. Do not use more than 12 V on the power jack. Ask an adult for batteries.

Keep it tidy
Short wires and a neat breadboard help you find mistakes quickly.

WHEN IT DOES NOT WORK (IT HAPPENS TO EVERY MAKER!)

1. **Read the red message** at the bottom of the IDE. It often tells the line with the mistake.
2. **Check the board and port** in the Tools menu.
3. **Check every wire.** Is each leg in the correct hole? Is GND connected?
4. **Check the code.** Missing semicolon? Missing brace? A wrong pin number?
5. **Test one small part at a time.** Then add the next part.
6. **Take a break and try again.** Ask a friend or a teacher. A fresh look finds the mistake.

FUN FACT
Mistakes are not failure. Every engineer in the world has debugged a circuit. **Debugging** means finding and fixing a problem. You are becoming a real engineer when you debug!

SECTION 2 | LEVEL: BEGINNER

Basic Projects

Light, sound and knobs. Your first real circuits.



Now you start to build! Each project is small and teaches one new idea. By the end of this section you can print messages, blink and fade lights, read buttons and knobs, make music and sense light.

WHAT YOU WILL LEARN

- Send messages with the Serial Monitor
- Control LEDs with digital and PWM pins
- Read a button and a knob (potentiometer)
- Make sounds with a buzzer
- Use a light sensor to decide automatically

PROJECTS IN THIS SECTION

No.	Project	New skill	Level	Time
01	**Hello World!**	Serial Monitor	★☆☆	15 min
02	**LED Blink**	Digital Output	★☆☆	20 min
03	**Traffic Light**	Digital Output	★☆☆	25 min
04	**Button Controls LED**	Digital Input	★☆☆	25 min
05	**Buzzer Melody**	Sound Output	★★☆	25 min
06	**Knob Reader**	Analog Input	★★☆	25 min
07	**Fading LED**	PWM Output	★★☆	20 min
08	**Night Light (LDR)**	Light Sensor	★★☆	30 min
09	**RGB Colour Mixer**	PWM Output	★★☆	30 min

ROBO SAYS
Build one project at a time. When it works, do the challenge at the end. That is how you really learn.

01 Hello World!

Serial Monitor | 15 min

WHAT IS IT?

Your Arduino can talk to your computer! In this first project, the board sends a message through the USB cable. You read the message on the screen. Every great maker starts here.

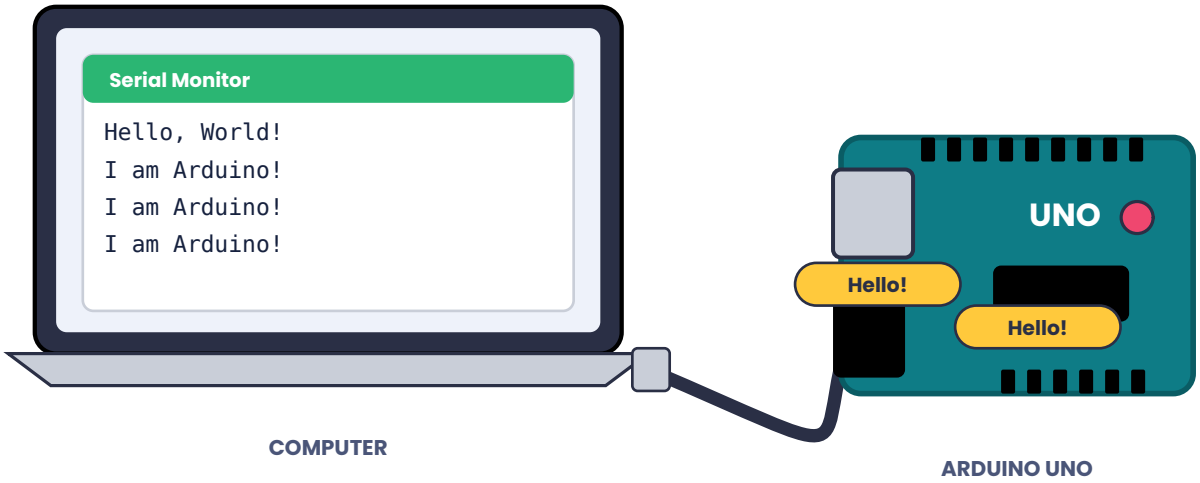
OBJECTIVE

Send your first message from Arduino to the computer and see it in the Serial Monitor.

HARDWARE YOU NEED

- **Arduino UNO** x1
- **USB cable** x1
- **Computer with IDE** x1

HOW IT FITS TOGETHER



i No wires or parts are needed. The USB cable gives power and carries the message.

STEP-BY-STEP PROCEDURE

1. Connect the Arduino to your computer with the USB cable.
2. Open the Arduino IDE. Choose **Tools > Board > Arduino Uno**.
3. Choose **Tools > Port** and pick the port that says **Arduino Uno**.
4. Type the code below. You can also copy it from the booklet page.
5. Click the **Upload** button (the arrow). Wait for "Done uploading".
6. Click the **Serial Monitor** button (magnifier icon, top right).
7. At the bottom right, set the speed to **9600 baud**. Now read your messages!

THE CODE (TYPE IT OR COPY IT)

```
void setup() {
  Serial.begin(9600); // start the talking line
  Serial.println("Hello, World!");
}

void loop() {
  Serial.println("I am Arduino!");
  delay(1000); // wait 1 second
}
```

HOW IT WORKS

- **Serial.begin(9600)** opens the talking line at speed 9600.
- **Serial.println()** sends words and then starts a new line.
- **delay(1000)** waits 1000 milliseconds. That is 1 second.

TRY IT YOURSELF
Change the message to your own name. Then change delay(1000) to delay(200). What happens?

ROBO SAYS
If you see strange symbols, the speed is wrong. Set the Serial Monitor to 9600 baud.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

02 LED Blink

Digital Output | 20 min

WHAT IS IT?

An LED is a tiny light. In this project the Arduino turns an LED on and off again and again. This is the "Hello World" of hardware. It shows how a program can control the real world.

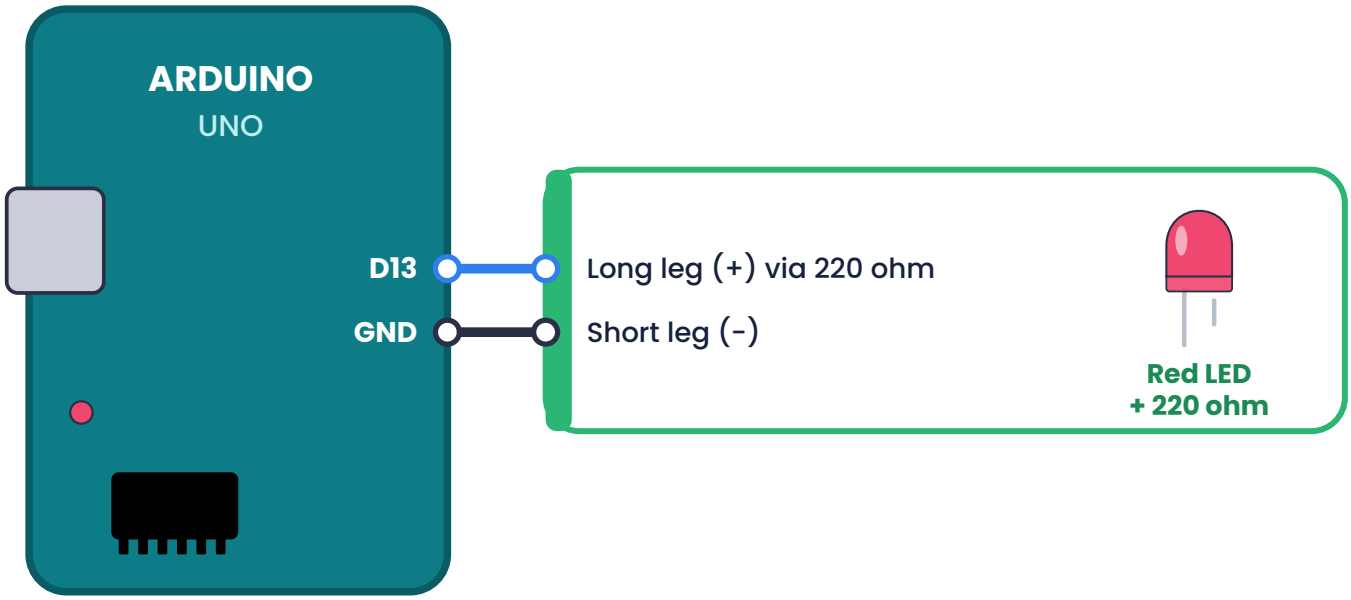
OBJECTIVE

Control one LED with the pin D13 using digitalWrite() and delay().

HARDWARE YOU NEED

- Arduino UNO x1
- LED (red) x1
- Resistor 220 ohm x1
- Breadboard x1
- Jumper wires x2

CONNECTION DIAGRAM



i The long leg of the LED is positive (+). Always put the resistor in the line. It protects the LED.

STEP-BY-STEP PROCEDURE

1. Put the LED on the breadboard. Keep the long leg and short leg in different rows.
2. Connect one end of the 220 ohm resistor to the long leg of the LED.
3. Use a jumper wire to join the other end of the resistor to **pin D13**.
4. Use another jumper wire to join the short leg of the LED to **GND**.
5. Type the code below and click **Upload**.
6. Watch the LED blink every second!

THE CODE (TYPE IT OR COPY IT)

```
const int ledPin = 13; // the LED is on pin 13

void setup() {
  pinMode(ledPin, OUTPUT); // pin 13 sends power OUT
}

void loop() {
  digitalWrite(ledPin, HIGH); // LED ON
  delay(1000); // wait 1 second
  digitalWrite(ledPin, LOW); // LED OFF
  delay(1000); // wait 1 second
}
```

HOW IT WORKS

- **pinMode(pin, OUTPUT)** tells Arduino this pin will give out power.
- **HIGH** means 5 volts (ON). **LOW** means 0 volts (OFF).
- The board also has a tiny LED already on pin 13. It blinks too!

TRY IT YOURSELF
Make the LED blink fast (100 ms). Then make it show the SOS signal: 3 short, 3 long, 3 short.

ROBO SAYS
If the LED does not light up, turn it around. LEDs only work in one direction.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

03 Traffic Light

Digital Output | 25 min

WHAT IS IT?

Build a small traffic light with three LEDs: red, yellow and green. The Arduino switches them in the right order, just like the lights on a real road.

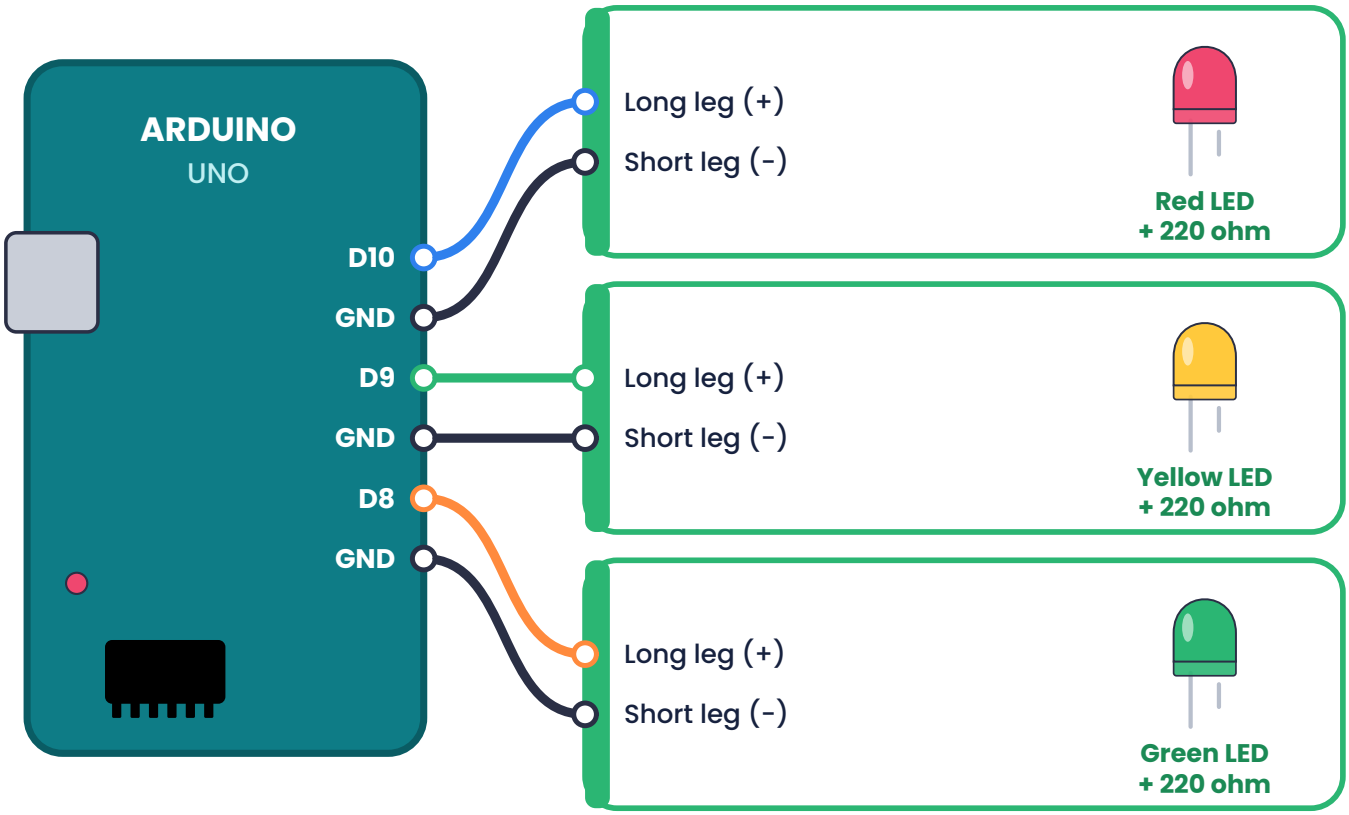
OBJECTIVE

Control three LEDs and learn how to plan a time order (a sequence).

HARDWARE YOU NEED

- Arduino UNO — x1
- Red LED — x1
- Yellow LED — x1
- Green LED — x1
- Resistor 220 ohm — x3
- Breadboard — x1

CONNECTION DIAGRAM



Use one 220 ohm resistor with each LED. All three short legs can share the same GND line on the breadboard.

STEP-BY-STEP PROCEDURE

1. Place the three LEDs side by side on the breadboard: red, yellow, green.
2. Put a 220 ohm resistor on the long leg of each LED.
3. Join the resistors to pins **D10** (red), **D9** (yellow) and **D8** (green).
4. Join all three short legs to the **GND** rail. Connect the rail to Arduino GND.
5. Upload the code and watch the lights change.

THE CODE (TYPE IT OR COPY IT)

```
const int redPin = 10;
const int yellowPin = 9;
const int greenPin = 8;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
}

void loop() {
  digitalWrite(redPin, HIGH); // STOP
  delay(4000);
  digitalWrite(redPin, LOW);

  digitalWrite(greenPin, HIGH); // GO
  delay(4000);
  digitalWrite(greenPin, LOW);

  digitalWrite(yellowPin, HIGH); // GET READY
  delay(1500);
  digitalWrite(yellowPin, LOW);
}
```

HOW IT WORKS

- Each LED has its own pin, so the code can control each one alone.
- A **const int** is a name for a number that never changes. It makes code easy to read.
- Only one light is ON at a time. We switch one OFF before we switch the next ON.

TRY IT YOURSELF
Add a 4th step: make the yellow light blink 3 times before the red light turns on.

ROBO SAYS
Draw the order on paper first: red, green, yellow, red... A plan makes coding easy.

04 Button Controls LED

Digital Input | 25 min

WHAT IS IT?

Until now the Arduino worked alone. Now you will give it an input! When you press a push button, the LED turns on. When you let go, it turns off.

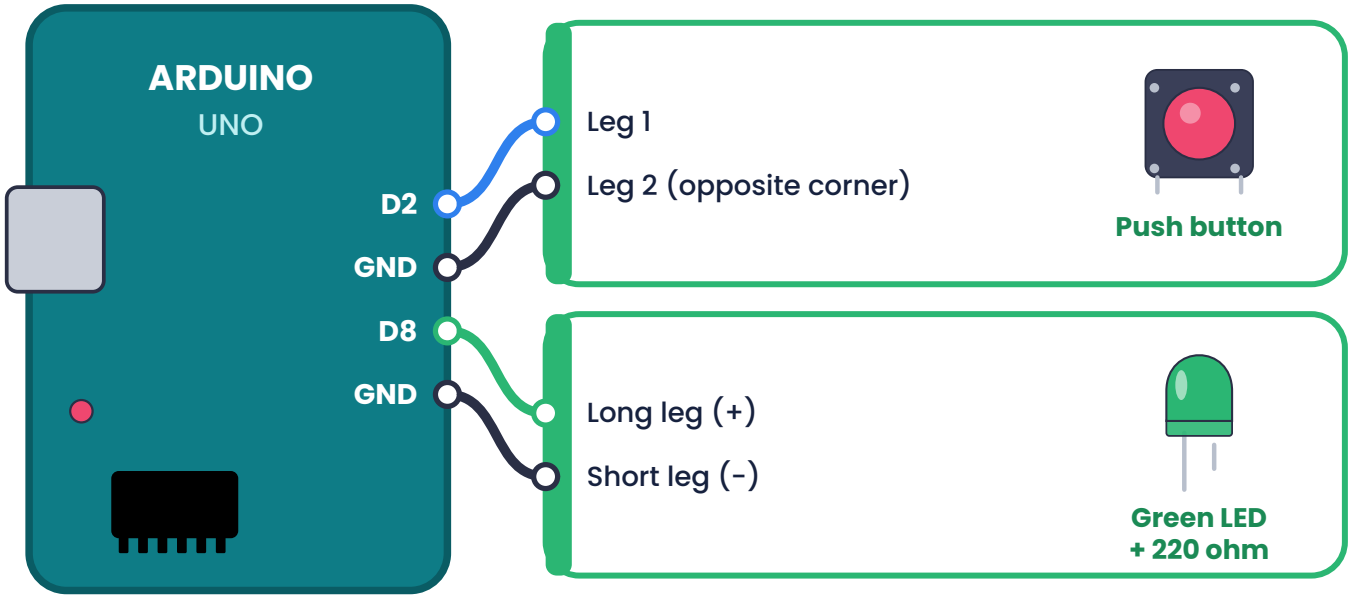
OBJECTIVE

Read a push button with digitalRead() and use if / else to make a decision.

HARDWARE YOU NEED

- Arduino UNO — x1
- Push button — x1
- LED (green) — x1
- Resistor 220 ohm — x1
- Breadboard — x1
- Jumper wires — x4

CONNECTION DIAGRAM



i We use INPUT_PULLUP. The pin reads HIGH when the button is free, and LOW when you press it.

STEP-BY-STEP PROCEDURE

1. Place the push button across the middle gap of the breadboard.
2. Join one leg of the button to **D2**. Join the opposite leg to **GND**.
3. Build the LED circuit like in Project 2, but use **pin D8**.
4. Upload the code.
5. Press the button. The LED turns ON. Let go. It turns OFF.
6. Open the Serial Monitor to see the button state too.

THE CODE (TYPE IT OR COPY IT)

```
const int buttonPin = 2;
const int ledPin = 8;

void setup() {
  pinMode(buttonPin, INPUT_PULLUP); // button input
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int state = digitalRead(buttonPin); // read the button

  if (state == LOW) { // pressed
    digitalWrite(ledPin, HIGH);
    Serial.println("Button: PRESSED");
  } else { // not pressed
    digitalWrite(ledPin, LOW);
    Serial.println("Button: free");
  }
  delay(100);
}
```

HOW IT WORKS

- **digitalRead()** checks if a pin is HIGH or LOW.
- **INPUT_PULLUP** keeps the pin HIGH until the button joins it to GND.
- **if / else** lets the Arduino choose between two actions.

TRY IT YOURSELF
Change the code so that each press toggles the LED: press once for ON, press again for OFF.

ROBO SAYS
Pressed = LOW may feel backwards. This is normal with INPUT_PULLUP.

05 Buzzer Melody

Sound Output | 25 min

WHAT IS IT?

A buzzer makes sound when it moves air very fast. Different speeds (frequencies) make different musical notes. Your Arduino will play the notes Do Re Mi and a small song.

OBJECTIVE

Make sound with tone() and use arrays to store a list of notes.

HARDWARE YOU NEED



Arduino UNO x1



Passive buzzer x1

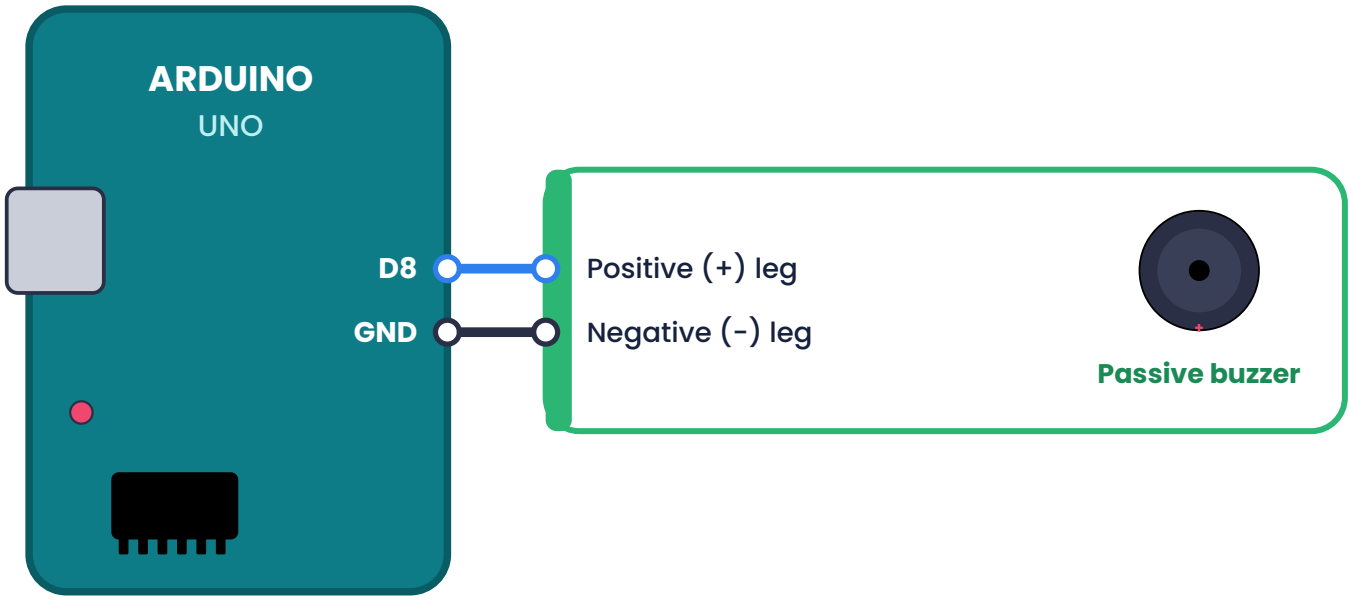


Breadboard x1



Jumper wires x2

CONNECTION DIAGRAM



i A passive buzzer plays different notes. An active buzzer plays only one beep sound.

STEP-BY-STEP PROCEDURE

1. Put the buzzer on the breadboard. The + sign is on the top of the buzzer.
2. Join the + leg to **pin D8**.
3. Join the other leg to **GND**.
4. Upload the code. First you hear the scale, then the song.
5. Change the numbers to make your own tune.

THE CODE (TYPE IT OR COPY IT)

```
const int buzzer = 8;

// notes in Hertz (Hz): C D E F G A B C
int scale[] = {262, 294, 330, 349, 392, 440, 494, 523};

// Twinkle Twinkle: C C G G A A G
int song[] = {262, 262, 392, 392, 440, 440, 392};

void setup() {
  pinMode(buzzer, OUTPUT);
}

void loop() {
  for (int i = 0; i < 8; i++) { // play the scale
    tone(buzzer, scale[i], 300);
    delay(400);
  }
  delay(1000);

  for (int i = 0; i < 7; i++) { // play the song
    tone(buzzer, song[i], 400);
    delay(500);
  }
  delay(2000);
}
```

HOW IT WORKS

- **tone(pin, frequency, time)** makes a sound. A big frequency makes a high note.
- An **array** is a list. **scale[0]** is the first number, **scale[1]** is the second.
- A **for loop** repeats the lines inside it. Here it plays each note one by one.

? TRY IT YOURSELF
Add more notes to finish the song: F F E E D D C. Can you play Happy Birthday?

i ROBO SAYS
Frequency is how many times the buzzer shakes in one second. 440 Hz is the note A.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

06 Knob Reader

Analog Input | 25 min

WHAT IS IT?

A potentiometer is a knob. When you turn it, its value changes smoothly. The Arduino reads the knob with an analog pin and shows the number on the screen.

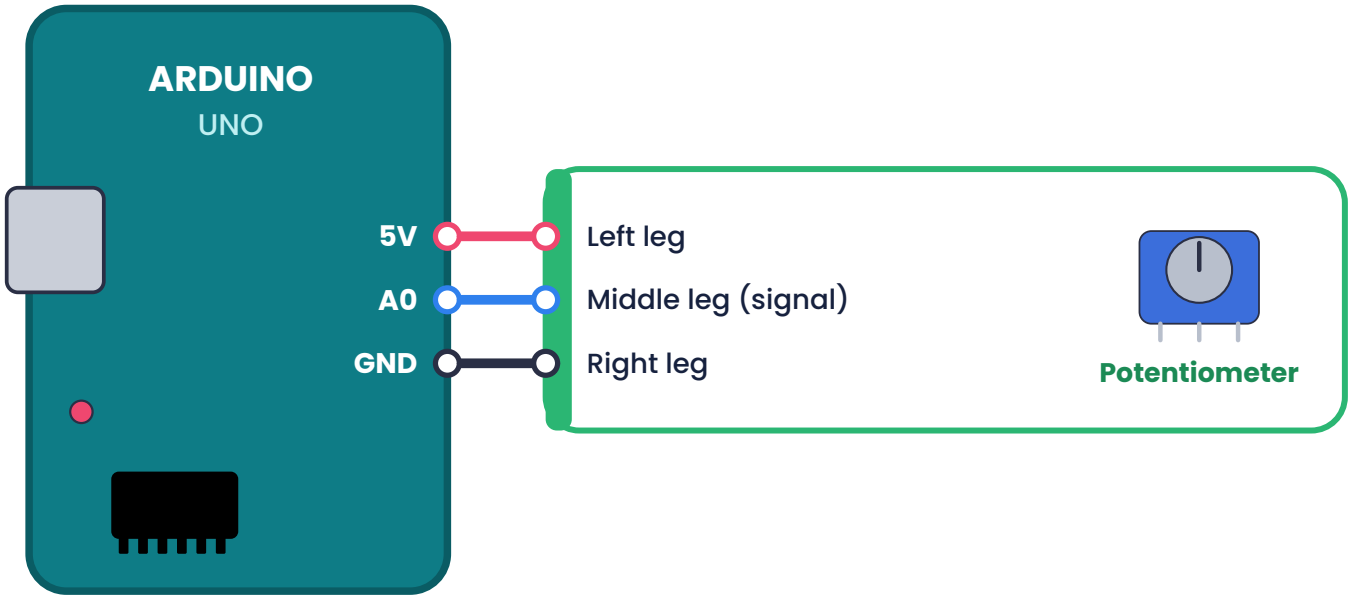
OBJECTIVE

Read an analog value (0 to 1023) and change it to a percent with map().

HARDWARE YOU NEED

- Arduino UNO — x1
- Potentiometer 10k — x1
- Breadboard — x1
- Jumper wires — x3

CONNECTION DIAGRAM



i The middle leg is the wiper. It gives a voltage between 0 V and 5 V, depending on the knob.

STEP-BY-STEP PROCEDURE

1. Put the potentiometer on the breadboard. The three legs must be in three different rows.
2. Join the left leg to **5V** and the right leg to **GND**.
3. Join the middle leg to **A0**.
4. Upload the code and open the Serial Monitor.
5. Turn the knob slowly and watch the numbers change.
6. Try **Tools > Serial Plotter** to see a moving line graph!

THE CODE (TYPE IT OR COPY IT)

```
const int knobPin = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int value = analogRead(knobPin); // 0 to 1023
  int percent = map(value, 0, 1023, 0, 100); // change to 0-100

  Serial.print("Value: ");
  Serial.print(value);
  Serial.print(" Percent: ");
  Serial.print(percent);
  Serial.println("%");
  delay(200);
}
```

HOW IT WORKS

- **analogRead()** measures a voltage and gives a number from 0 to 1023.
- 0 means 0 volts. 1023 means 5 volts.
- **map(value, 0, 1023, 0, 100)** changes one range of numbers into another.

TRY IT YOURSELF
Print the voltage too. Hint: voltage = value x 5.0 / 1023.0

ROBO SAYS
Many future projects use this knob to control brightness, speed and angle.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

07 Fading LED

PWM Output | 20 min

WHAT IS IT?

Can an LED be half bright? Yes! The Arduino switches the LED on and off very fast. This trick is called PWM. Your eyes see a smooth glow, so the LED seems to breathe.

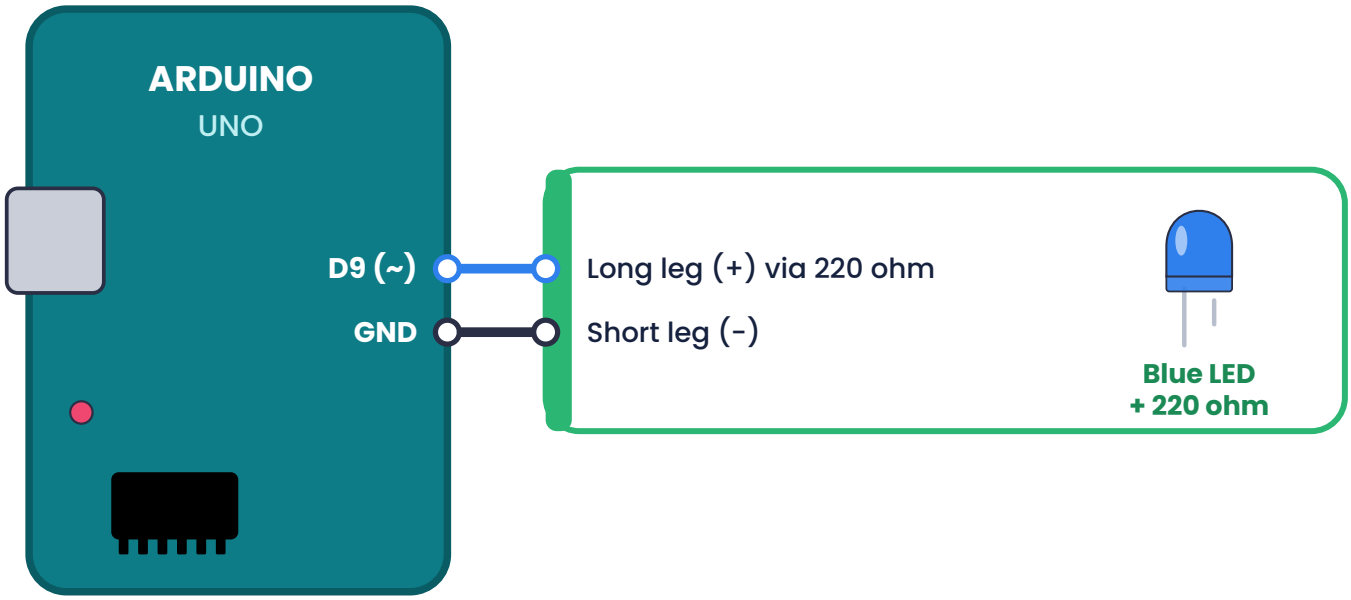
OBJECTIVE

Learn PWM and use analogWrite() on a pin with the ~ sign.

HARDWARE YOU NEED

- Arduino UNO x1
- LED (blue) x1
- Resistor 220 ohm x1
- Breadboard x1
- Jumper wires x2

CONNECTION DIAGRAM



i Only pins with a ~ sign can do PWM on the Uno: 3, 5, 6, 9, 10 and 11.

STEP-BY-STEP PROCEDURE

1. Build the same LED circuit as in Project 2.
2. This time use **pin D9**. It has a ~ sign.
3. Upload the code.
4. Watch the LED slowly get brighter and then dimmer.

THE CODE (TYPE IT OR COPY IT)

```
const int ledPin = 9; // a PWM pin (~)

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  for (int b = 0; b <= 255; b++) { // brighter
    analogWrite(ledPin, b);
    delay(8);
  }
  for (int b = 255; b >= 0; b--) { // dimmer
    analogWrite(ledPin, b);
    delay(8);
  }
}
```

HOW IT WORKS

- **analogWrite(pin, 0 to 255)**: 0 is OFF, 255 is full bright, 127 is half bright.
- PWM means Pulse Width Modulation. The longer the ON time, the brighter the light.
- A **for loop** counts up (b++) and then down (b--).

TRY IT YOURSELF
Use Project 6: let the knob control the LED brightness. Use map(value, 0, 1023, 0, 255).

ROBO SAYS
Smaller delay = faster fade. Try delay(2) and delay(20).

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

08 Night Light (LDR)

Light Sensor | 30 min

WHAT IS IT?

An LDR is a light-sensing resistor. In the dark its value goes down. You will build a night light that turns on by itself when the room becomes dark.

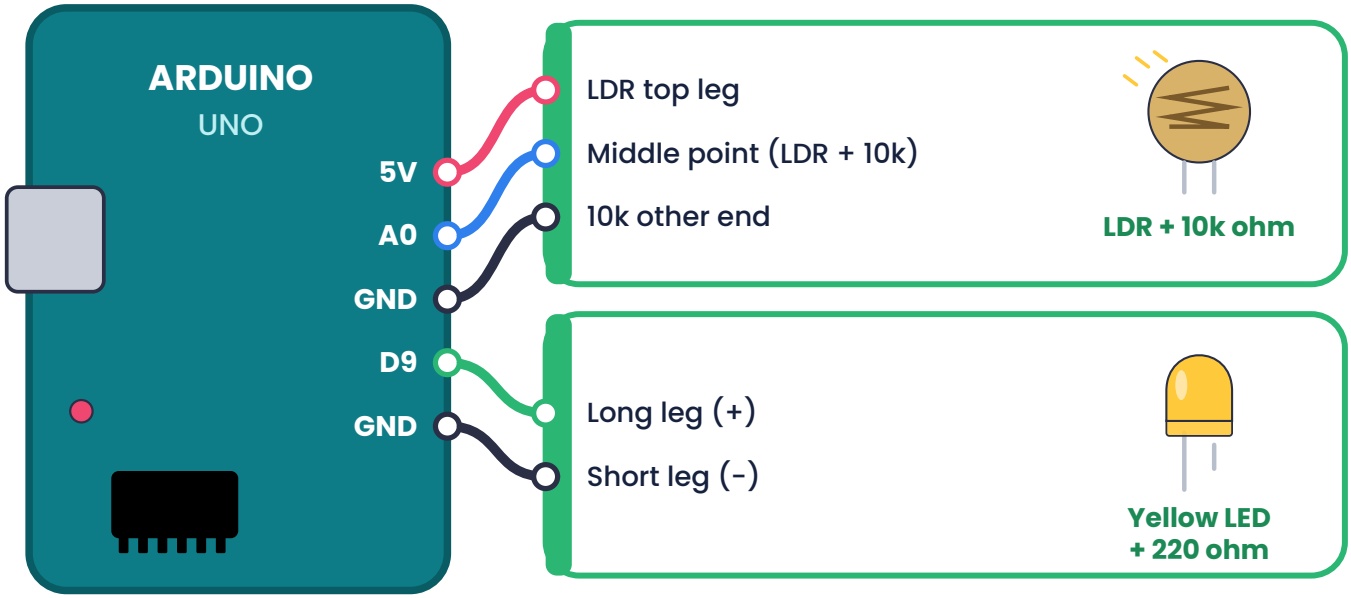
OBJECTIVE

Use a sensor and a threshold number to make an automatic decision.

HARDWARE YOU NEED

- Arduino UNO x1
- LDR sensor x1
- Resistor 10k ohm x1
- LED (yellow) x1
- Resistor 220 ohm x1
- Breadboard x1

CONNECTION DIAGRAM



The LDR and the 10k resistor make a voltage divider. A0 reads the point between them.

STEP-BY-STEP PROCEDURE

1. Put the LDR and the 10k resistor in the same row, one after the other.
2. Join the free end of the LDR to **5V** and the free end of the resistor to **GND**.
3. Join the point where they meet to **A0**.
4. Build the LED circuit on **pin D9** like before.
5. Upload the code. Open the Serial Monitor and cover the LDR with your hand.
6. Note the numbers in light and in dark. Change **dark** in the code to fit your room.

THE CODE (TYPE IT OR COPY IT)

```
const int ldrPin = A0;
const int ledPin = 9;
int dark = 400; // below this number = dark

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int light = analogRead(ldrPin);
  Serial.println(light);

  if (light < dark) {
    digitalWrite(ledPin, HIGH); // dark: lamp ON
  } else {
    digitalWrite(ledPin, LOW); // bright: lamp OFF
  }
  delay(200);
}
```

HOW IT WORKS

- More light = bigger number. Less light = smaller number (with this wiring).
- The number **dark** is a **threshold**. It is the line between day and night.
- Your room may be different. Test it and choose your own number.

? TRY IT YOURSELF
Make the lamp smooth. Use analogWrite so the LED gets brighter as the room gets darker.

i ROBO SAYS
Do not point a torch directly at the LDR for a long time. Use normal room light.

09 RGB Colour Mixer

PWM Output | 30 min

WHAT IS IT?

An RGB LED has three tiny lights inside: red, green and blue. Mix them in different amounts and you can make any colour, like a painter mixing paint.

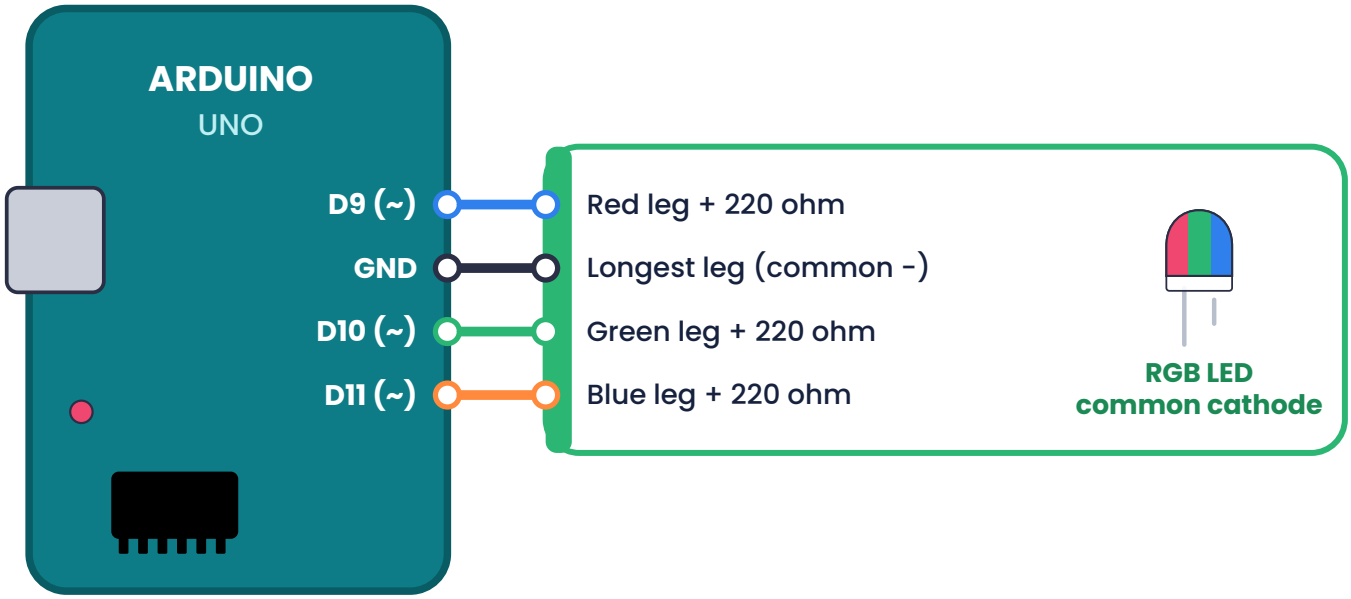
OBJECTIVE

Control three PWM pins at the same time and write your own function.

HARDWARE YOU NEED

- Arduino UNO x1
- RGB LED common cath x1
- Resistor 220 ohm x3
- Breadboard x1
- Jumper wires x5

CONNECTION DIAGRAM



i The longest leg is the common leg. If your LED is common anode, join it to 5V and use 255 - value in the code.

STEP-BY-STEP PROCEDURE

1. Find the longest leg of the RGB LED. This is the common leg.
2. Put the LED on the breadboard. Join the longest leg to **GND**.
3. Add a 220 ohm resistor to each of the other three legs.
4. Join the red leg to **D9**, green to **D10** and blue to **D11**.
5. Upload the code and watch the colours change.

THE CODE (TYPE IT OR COPY IT)

```
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
}

// our own function: mix a colour (0 to 255 each)
void setColor(int r, int g, int b) {
  analogWrite(redPin, r);
  analogWrite(greenPin, g);
  analogWrite(bluePin, b);
}

void loop() {
  setColor(255, 0, 0); delay(800); // red
  setColor(0, 255, 0); delay(800); // green
  setColor(0, 0, 255); delay(800); // blue
  setColor(255, 255, 0); delay(800); // yellow
  setColor(0, 255, 255); delay(800); // cyan
  setColor(255, 0, 255); delay(800); // purple
  setColor(255, 255, 255); delay(800); // white
}
```

HOW IT WORKS

- A **function** is a small named program. **setColor(r, g, b)** saves us from writing 3 lines each time.
- Red + Green = Yellow. Green + Blue = Cyan. Red + Blue = Purple. All three = White.
- Each colour value goes from 0 (off) to 255 (full).

TRY IT YOURSELF
Use three potentiometers (one for each colour) to build a real colour mixer.

ROBO SAYS
Look at the LED from a little distance. The three colours blend into one.

MY LAB NOTES

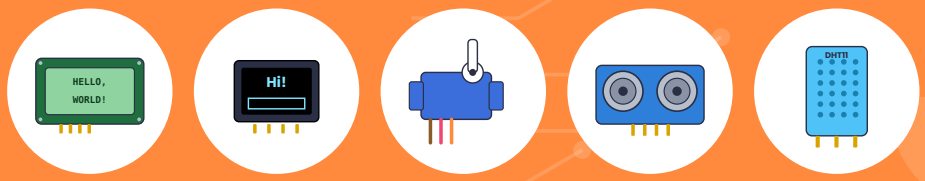
☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

SECTION 3 | LEVEL: EXPLORER

Intermediate Projects

Screens, sensors and motion.



You know the basics. Now your projects can show words, move and sense the world. You will also install libraries, which are boxes of ready-made code from other makers.

WHAT YOU WILL LEARN

- Show text and shapes on LCD and OLED screens
- Use libraries to talk to new parts
- Move a servo to an exact angle
- Measure distance, temperature, motion and soil moisture
- Combine many parts in one program

PROJECTS IN THIS SECTION

No.	Project	New skill	Level	Time
10	**LCD Message Board**	I2C Display	★★☆	30 min
11	**OLED Display**	I2C Display	★★☆	30 min
12	**Servo Sweep**	Motion Output	★★☆	25 min
13	**Servo Knob Control**	Input + Motion	★★☆	25 min
14	**Distance Meter**	Ultrasonic Sensor	★★★	35 min
15	**Weather on the LCD**	Temperature Sensor	★★☆	35 min
16	**Motion Alarm**	PIR Sensor	★★☆	30 min
17	**Plant Water Monitor**	Soil Sensor + OLED	★★★	40 min
18	**Smart Parking Helper**	Ultrasonic + LEDs + Buzzer	★★★	40 min

ROBO SAYS
Libraries are installed from **Sketch > Include Library > Manage Libraries**. You need the internet for this one time.

10 LCD Message Board

I2C Display | 30 min

WHAT IS IT?

An LCD screen can show letters and numbers. This one has 2 lines with 16 letters each. It talks to the Arduino with only 4 wires. This talking method is called I2C.

OBJECTIVE

Connect an I2C LCD, install a library and print text on two lines.

HARDWARE YOU NEED



Arduino UNO x1

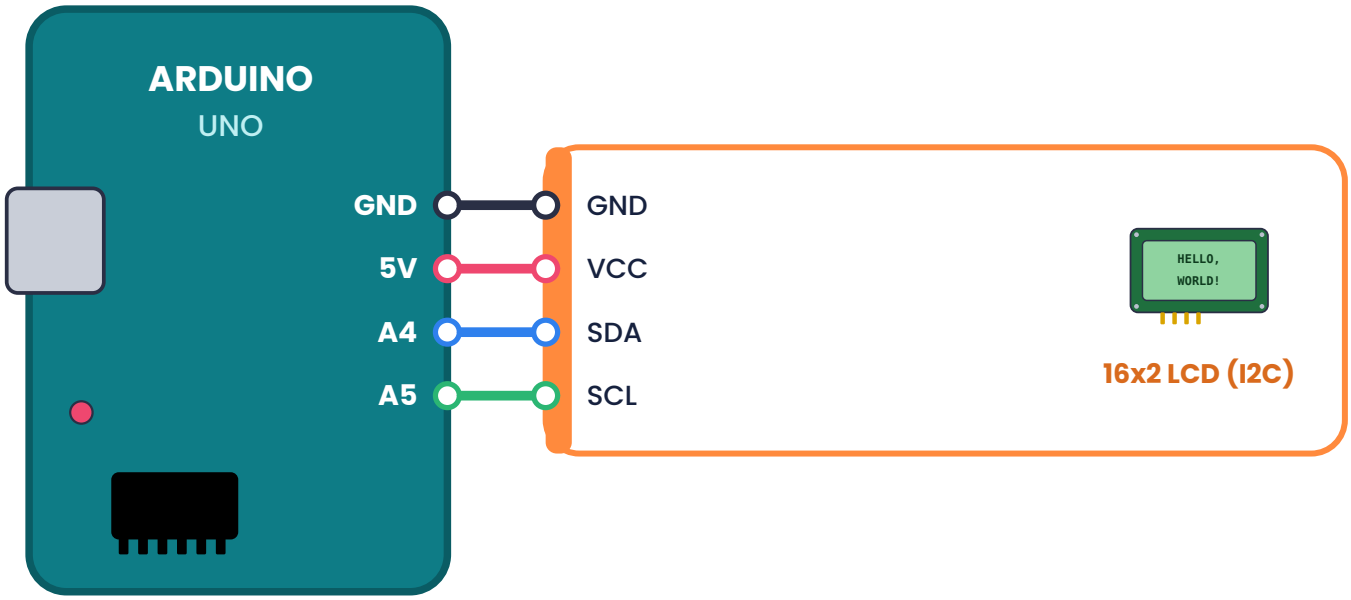


16x2 LCD with I2C x1



Jumper wires x4

CONNECTION DIAGRAM



i On the Arduino Uno, I2C always uses A4 (SDA, data) and A5 (SCL, clock).

STEP-BY-STEP PROCEDURE

1. Connect the 4 wires from the LCD to the Arduino as shown in the diagram.
2. Install the library: **Sketch > Include Library > Manage Libraries**. Search **LiquidCrystal I2C** and install the one by Frank de Brabander.
3. Upload the code.
4. If you see only white boxes, turn the small blue screw on the back of the LCD. This changes the contrast.
5. If nothing appears, change **0x27** to **0x3F** in the code. Some screens use that address.

THE CODE (TYPE IT OR COPY IT)

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2); // address, columns, rows

void setup() {
  lcd.init(); // start the screen
  lcd.backlight(); // turn on the light
  lcd.setCursor(0, 0); // column 0, row 0
  lcd.print("Hello, World!");
  lcd.setCursor(0, 1); // column 0, row 1
  lcd.print("Seconds: ");
}

void loop() {
  lcd.setCursor(9, 1);
  lcd.print(millis() / 1000); // time since start
  delay(500);
}
```

HOW IT WORKS

- **#include** brings a library into your program. A library is a box of ready-made code.
- **setCursor(column, row)** chooses where the next text starts.
- **millis()** gives the milliseconds since the Arduino started.

? TRY IT YOURSELF
Show your name on line 1 and a counter that goes up each second on line 2.

i ROBO SAYS
An I2C address is like a house number. Many screens use 0x27 or 0x3F.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

11 OLED Display

I2C Display | 30 min

WHAT IS IT?

An OLED screen is small but bright and sharp. Every dot (pixel) makes its own light. You can write text and draw shapes on it, just like a tiny computer screen.

OBJECTIVE

Use the Adafruit libraries to write text and draw shapes on a 128x64 OLED.

HARDWARE YOU NEED



Arduino UNO x1

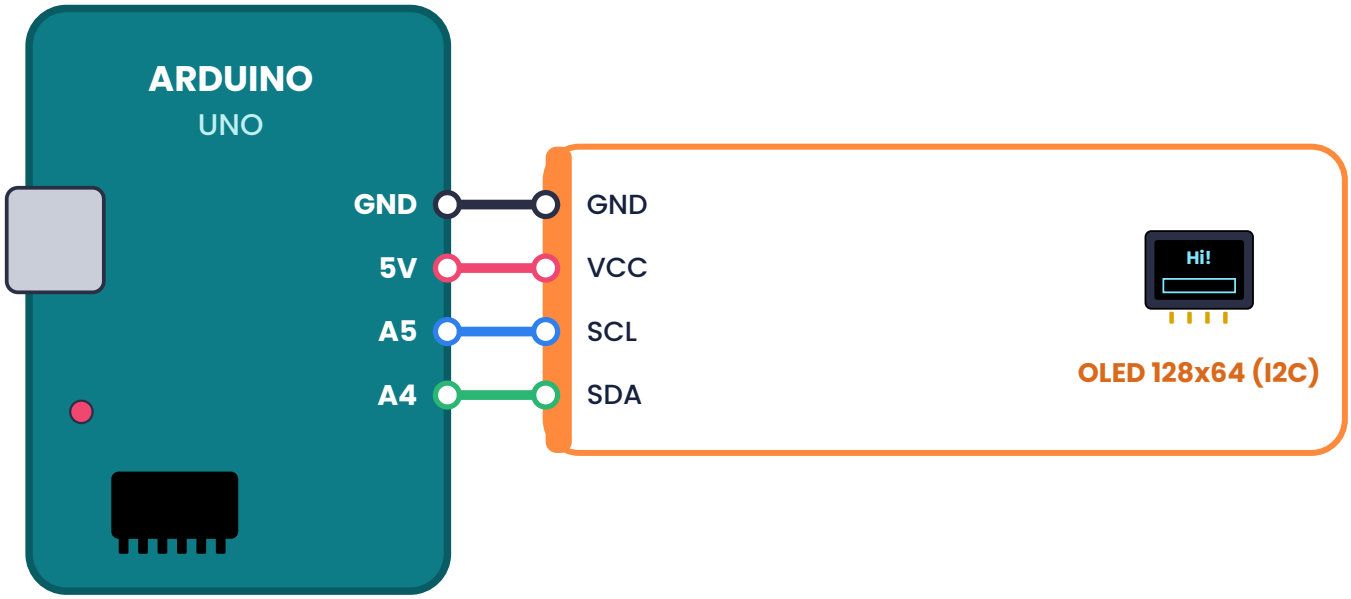


0.96 inch OLED I2C x1



Jumper wires x4

CONNECTION DIAGRAM



Pin order on OLED boards can be different. Read the tiny letters on your board before you connect.

STEP-BY-STEP PROCEDURE

1. Connect GND, VCC, SCL and SDA as shown. Check the pin names on your board.
2. Open **Manage Libraries**. Install **Adafruit SSD1306** and **Adafruit GFX Library**.
3. Upload the code.
4. The screen shows text, a box and a moving ball.
5. If the screen stays black, try the address **0x3D** instead of 0x3C.

THE CODE (TYPE IT OR COPY IT)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);
int x = 0; // ball position
int speedX = 3; // ball speed

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.setTextColor(SSD1306_WHITE);
}

void loop() {
  display.clearDisplay();

  display.setTextSize(2);
  display.setCursor(0, 0);
  display.println("Hello!");
  display.setTextSize(1);
  display.println("I am an OLED.");

  display.drawRect(0, 30, 128, 34, SSD1306_WHITE); // box
  display.fillCircle(x, 47, 6, SSD1306_WHITE); // ball

  x = x + speedX;
  if (x > 120 || x < 8) speedX = -speedX; // bounce

  display.display(); // send picture to the screen
  delay(30);
}
```

HOW IT WORKS

- Everything you draw goes into memory first. **display.display()** shows it on the screen.
- **clearDisplay()** wipes the old picture before the new one.
- When the ball touches the box edge, **speedX = -speedX** turns it around.

TRY IT YOURSELF
Draw your own face with drawCircle() and drawLine(). Make it blink.

ROBO SAYS
The screen is 128 pixels wide and 64 pixels high. The top-left corner is (0, 0).

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

12 Servo Sweep

Motion Output | 25 min

WHAT IS IT?

A servo motor turns to an exact angle between 0 and 180 degrees. Robot arms, doors and steering all use servos. Now you will make one sweep left and right.

OBJECTIVE

Control a servo with the Servo library using angles in degrees.

HARDWARE YOU NEED



Arduino UNO x1

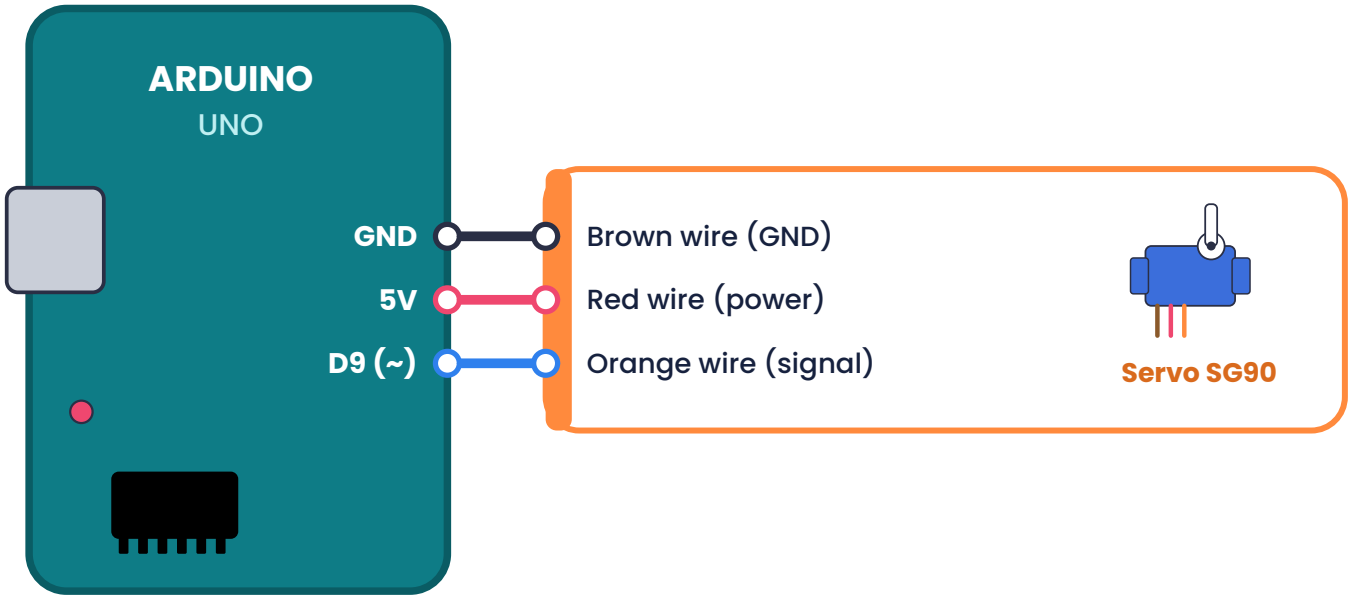


Servo SG90 x1



Jumper wires x3

CONNECTION DIAGRAM



i Servo wire colours: brown = GND, red = 5V, orange or yellow = signal.

STEP-BY-STEP PROCEDURE

1. Push the servo plug into three jumper wires. Match the colours.
2. Join brown to **GND**, red to **5V** and orange to **D9**.
3. Attach the white arm (horn) on the servo shaft.
4. Upload the code. The arm sweeps from 0 to 180 degrees and back.

THE CODE (TYPE IT OR COPY IT)

```
#include <Servo.h>

Servo myServo; // make a servo object

void setup() {
  myServo.attach(9); // signal wire on pin 9
}

void loop() {
  for (int angle = 0; angle <= 180; angle++) {
    myServo.write(angle); // turn to this angle
    delay(15);
  }
  for (int angle = 180; angle >= 0; angle--) {
    myServo.write(angle);
    delay(15);
  }
}
```

HOW IT WORKS

- **myServo.write(90)** turns the arm to the middle position.
- The delay(15) gives the servo time to reach each angle.
- A small servo can run from the Arduino 5V. Bigger servos need their own battery.

TRY IT YOURSELF
Make the servo jump to 0, 90 and 180 degrees with a 1 second wait between each.

ROBO SAYS
Do not force the arm by hand while the servo is on. You can break the gears.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

13 Servo Knob Control

Input + Motion | 25 min

WHAT IS IT?

Time to take control! Turn a knob and the servo follows your hand, like a robot arm. This project joins an input (the knob) with an output (the servo).

OBJECTIVE

Read an analog input and map it to a servo angle.

HARDWARE YOU NEED



Arduino UNO x1



Servo SG90 x1



Potentiometer 10k x1

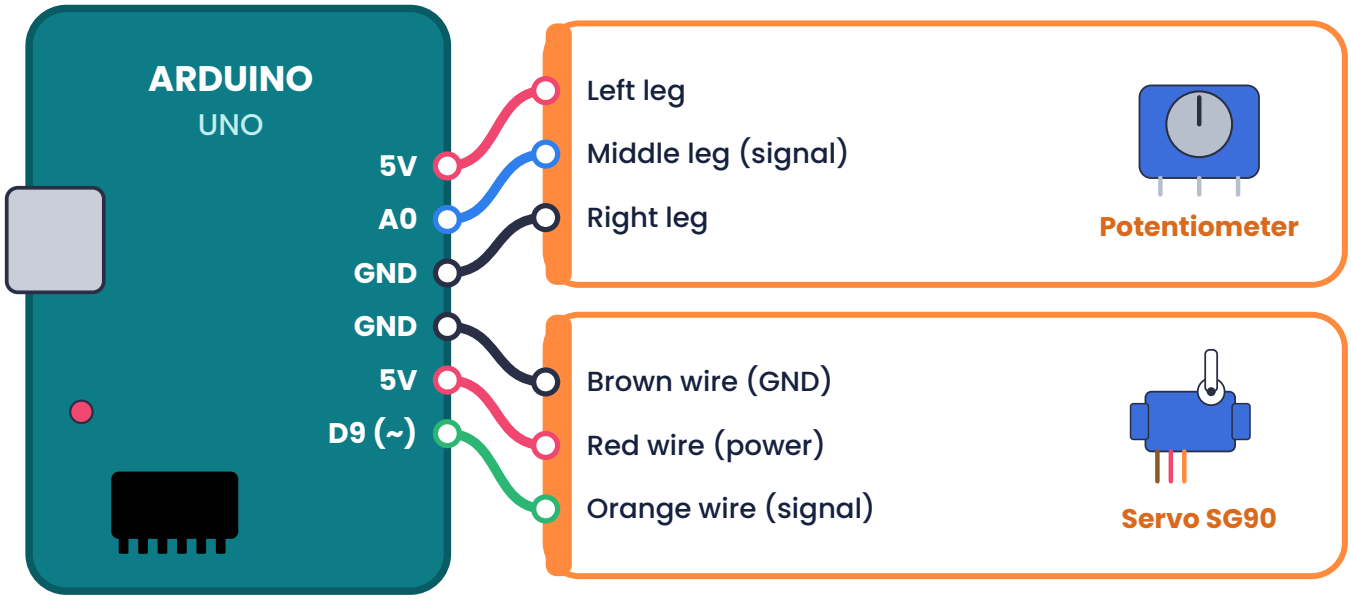


Breadboard x1



Jumper wires x6

CONNECTION DIAGRAM



i Both parts need 5V and GND. Use the breadboard rails to share them.

STEP-BY-STEP PROCEDURE

1. Connect the potentiometer like in Project 6. The middle leg goes to **A0**.
2. Connect the servo like in Project 12. The signal wire goes to **D9**.
3. Use the breadboard rails: one for 5V, one for GND, so both parts can share them.
4. Upload the code and turn the knob. The servo arm follows!

THE CODE (TYPE IT OR COPY IT)

```
#include <Servo.h>

Servo myServo;
const int knobPin = A0;

void setup() {
  myServo.attach(9);
}

void loop() {
  int value = analogRead(knobPin); // 0 to 1023
  int angle = map(value, 0, 1023, 0, 180); // 0 to 180
  myServo.write(angle);
  delay(15);
}
```

HOW IT WORKS

- The same **map()** from Project 6 now changes 0-1023 into 0-180 degrees.
- The loop runs hundreds of times a second, so the servo follows smoothly.
- This is a tiny **control system**: input, brain, output.

TRY IT YOURSELF
Add an LED on pin 8 that gets brighter when the angle is bigger.

ROBO SAYS
If the servo shakes, the power may be weak. Use a battery pack or a stronger USB port.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

14 Distance Meter

Ultrasonic Sensor | 35 min

WHAT IS IT?

Bats find their way with sound. The HC-SR04 sensor does the same. It sends a sound you cannot hear, waits for the echo, and the Arduino calculates the distance.

OBJECTIVE

Measure distance with an ultrasonic sensor using pulseIn() and a formula.

HARDWARE YOU NEED



Arduino UNO x1

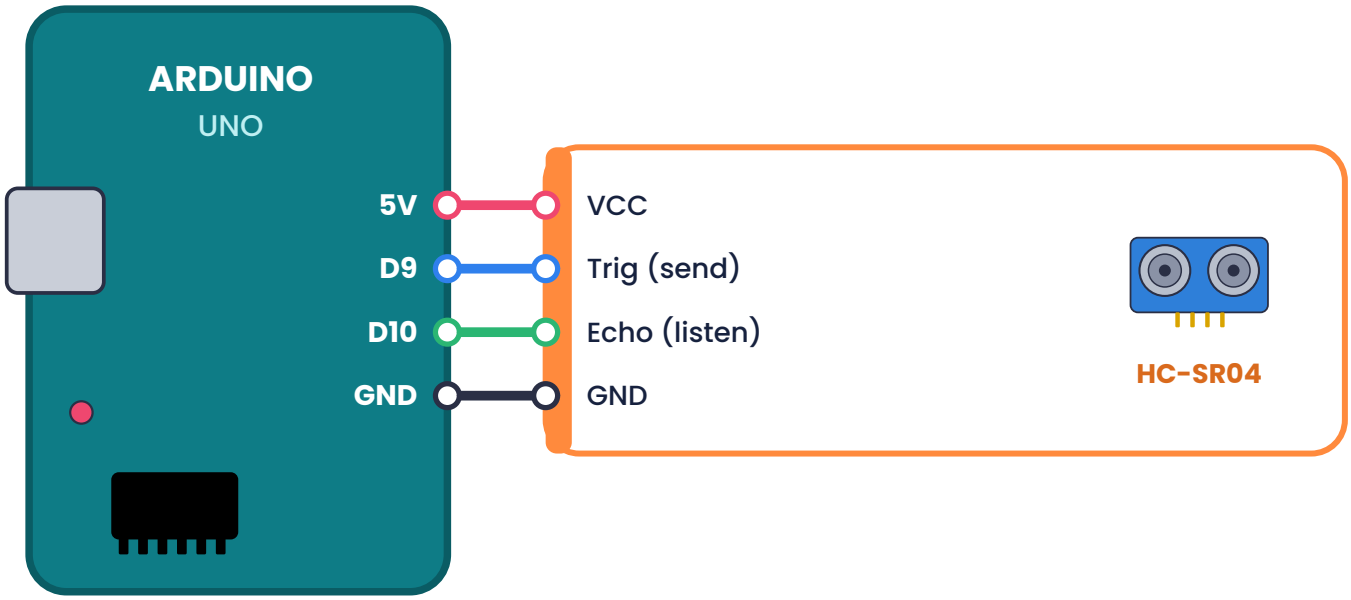


HC-SR04 sensor x1



Jumper wires x4

CONNECTION DIAGRAM



i Trig sends the sound. Echo listens. The sensor can see about 2 cm to 400 cm.

STEP-BY-STEP PROCEDURE

1. Connect the four pins of the sensor as shown in the diagram.
2. Upload the code and open the Serial Monitor.
3. Put your hand in front of the sensor. Move it closer and further.
4. Check the numbers with a ruler. Are they right?

THE CODE (TYPE IT OR COPY IT)

```
const int trigPin = 9;
const int echoPin = 10;

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(trigPin, LOW); // clean start
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); // send a short sound
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long time = pulseIn(echoPin, HIGH); // time of the echo
  float distance = time * 0.034 / 2; // in centimetres

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
  delay(300);
}
```

HOW IT WORKS

- Sound travels 0.034 cm every microsecond.
- The sound goes to the object and back. So we divide by 2.
- **distance = time x 0.034 / 2**

TRY IT YOURSELF
Turn an LED on when the distance is less than 10 cm. Use if (distance < 10).

ROBO SAYS
Soft things like cloth and sponge give weak echoes. Try a book or a wall.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

15 Weather on the LCD

Temperature Sensor | 35 min

WHAT IS IT?

The DHT11 sensor measures how hot the air is and how much water is in it (humidity). You will show both numbers on the LCD, like a small weather report.

OBJECTIVE

Read a digital sensor with a library and show the values on an LCD.

HARDWARE YOU NEED



Arduino UNO x1



DHT11 sensor x1

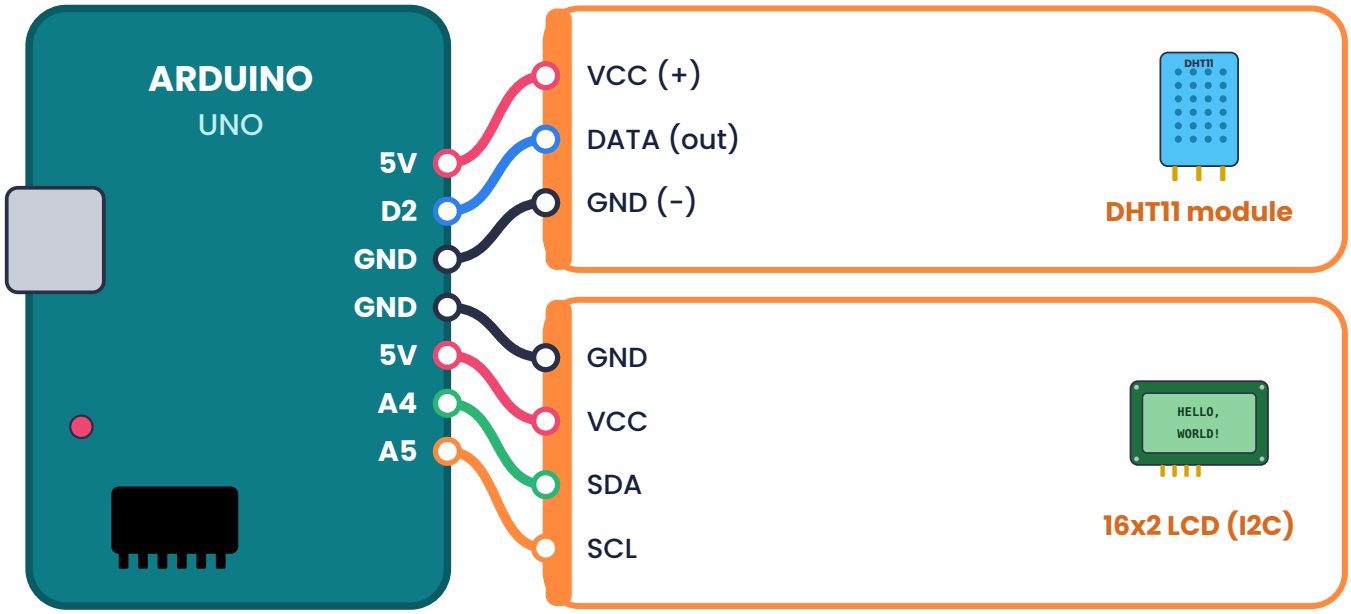


16x2 LCD with I2C x1



Jumper wires x7

CONNECTION DIAGRAM



i Use the 3-pin DHT11 module. It already has the small resistor on the board.

STEP-BY-STEP PROCEDURE

1. Connect the DHT11: VCC to 5V, DATA to **D2**, GND to GND.
2. Connect the LCD like in Project 10.
3. Install the library **DHT sensor library** by Adafruit. Install **Adafruit Unified Sensor** when asked.
4. Upload the code.
5. Breathe gently on the sensor. The humidity goes up!

THE CODE (TYPE IT OR COPY IT)

```
#include <DHT.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  lcd.init();
  lcd.backlight();
  dht.begin();
}

void loop() {
  float t = dht.readTemperature(); // in Celsius
  float h = dht.readHumidity(); // in percent

  lcd.clear();
  if (isnan(t) || isnan(h)) { // is the reading bad?
    lcd.print("Sensor error");
  } else {
    lcd.setCursor(0, 0);
    lcd.print("Temp: ");
    lcd.print(t, 1);
    lcd.print(" C");
    lcd.setCursor(0, 1);
    lcd.print("Humid: ");
    lcd.print(h, 0);
    lcd.print(" %");
  }
  delay(2000); // DHT11 needs 2 seconds between reads
}
```

HOW IT WORKS

- **readTemperature()** gives degrees Celsius. **readHumidity()** gives percent.
- **isnan()** checks if the number is "not a number". It protects us from a bad reading.
- **lcd.print(t, 1)** prints t with 1 number after the dot.

TRY IT YOURSELF
Show the temperature in Fahrenheit too. Formula: F = C x 1.8 + 32.

ROBO SAYS
DHT11 is slow. Wait 2 seconds between readings.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

16 Motion Alarm

PIR Sensor | 30 min

WHAT IS IT?

Your body gives out invisible heat. The PIR sensor can feel it when you move. Build an alarm that blinks and beeps when someone walks in front of it.

OBJECTIVE

Read a PIR motion sensor and trigger a buzzer and an LED.

HARDWARE YOU NEED



Arduino UNO x1



PIR motion sensor x1



Buzzer x1



LED (red) x1

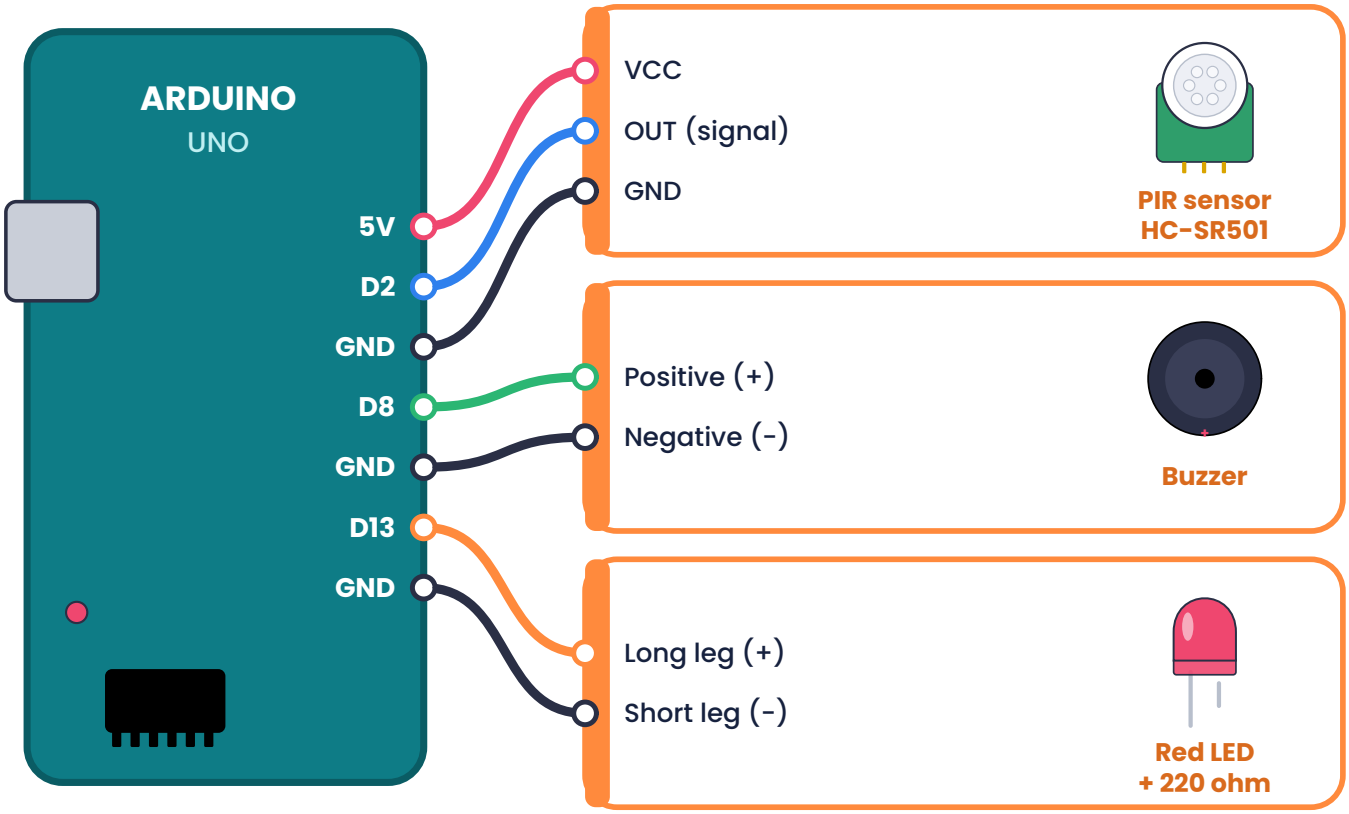


Resistor 220 ohm x1



Breadboard x1

CONNECTION DIAGRAM



After power-on, the PIR needs about 30 seconds to warm up. Stay still during this time.

STEP-BY-STEP PROCEDURE

1. Connect the PIR: VCC to 5V, OUT to **D2**, GND to GND.
2. Connect the buzzer to **D8** and GND.
3. Connect the LED with its resistor to **D13** and GND.
4. Upload the code. Wait 30 seconds.
5. Wave your hand in front of the sensor. The alarm starts!

THE CODE (TYPE IT OR COPY IT)

```
const int pirPin = 2;
const int buzzer = 8;
const int ledPin = 13;

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(buzzer, OUTPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
  Serial.println("Warming up... wait 30 seconds");
  delay(30000);
  Serial.println("Ready!");
}

void loop() {
  if (digitalRead(pirPin) == HIGH) { // motion found
    Serial.println("Motion detected!");
    digitalWrite(ledPin, HIGH);
    tone(buzzer, 1000); // start the beep
    delay(500);
    noTone(buzzer);
    digitalWrite(ledPin, LOW);
    delay(200);
  }
}
```

HOW IT WORKS

- The PIR output is HIGH when it feels motion, and LOW when all is quiet.
- **tone(pin, frequency)** without a time keeps beeping until **noTone()**.
- The two small screws on the PIR change the distance and the hold time.

TRY IT YOURSELF
Count how many times the alarm is triggered. Print the number on the Serial Monitor.

ROBO SAYS
PIR sensors work best in a room. Strong sunlight and hot air can confuse them.

17 Plant Water Monitor

Soil Sensor + OLED | 40 min

WHAT IS IT?

Plants cannot say "I am thirsty". Your Arduino can! A soil sensor measures how wet the soil is. The OLED shows the answer, and an LED warns you when the plant needs water.

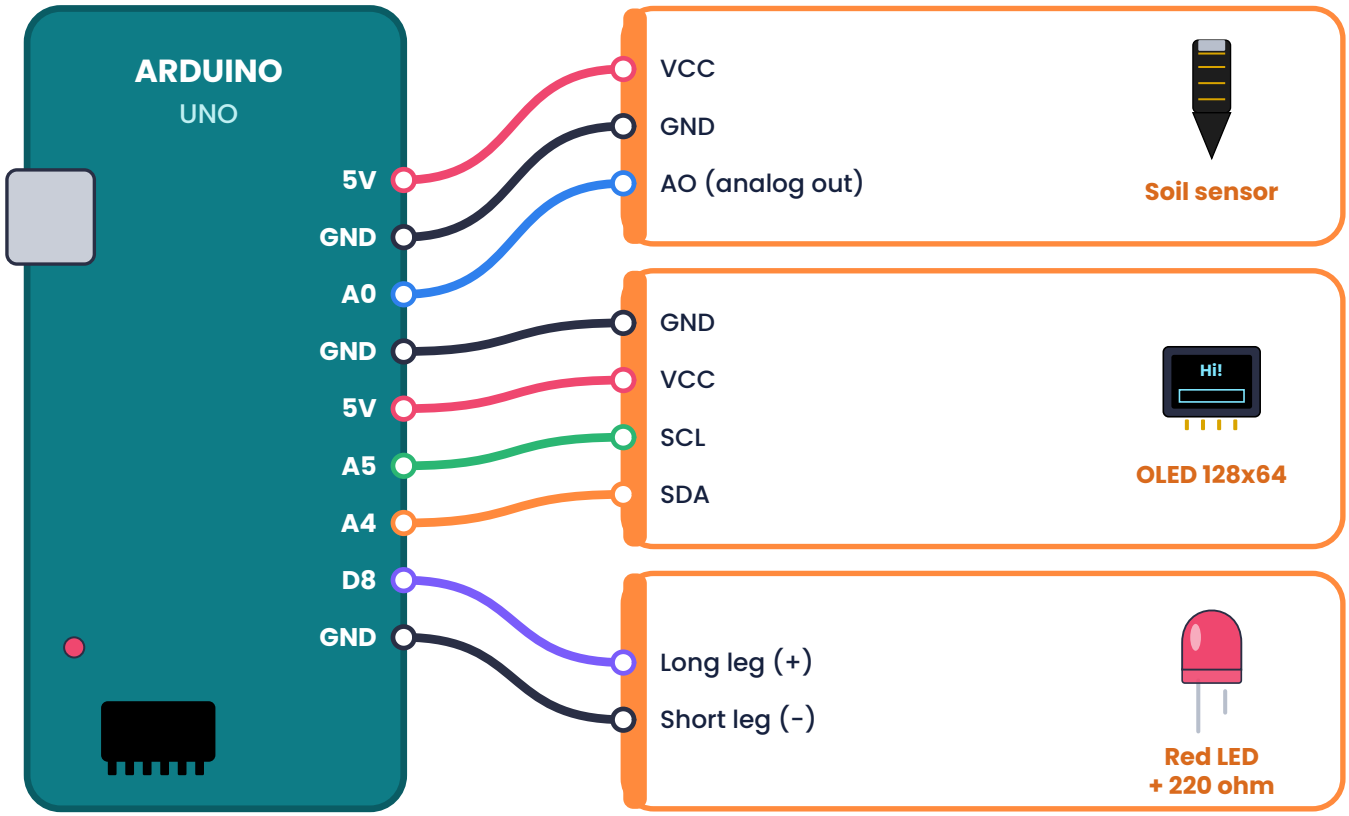
OBJECTIVE

Read a soil moisture sensor, make it a percent and show it with a bar on the OLED.

HARDWARE YOU NEED

- Arduino UNO x1
- Soil moisture sensor x1
- OLED I2C x1
- LED (red) x1
- Resistor 220 ohm x1
- Jumper wires x9

CONNECTION DIAGRAM



Use a capacitive soil sensor if you can. Plain two-prong sensors rust quickly in wet soil.

STEP-BY-STEP PROCEDURE

1. Connect the soil sensor, the OLED and the LED as shown.
2. Upload the code. Hold the sensor in dry air and note the number. This is **dryValue**.
3. Put the sensor in a glass of water (only up to the line). Note the number. This is **wetValue**.
4. Write both numbers in the code. Upload again.
5. Put the sensor in your plant pot. Water the plant and watch the bar grow!

THE CODE (TYPE IT OR COPY IT)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);
const int soilPin = A0;
const int ledPin = 8;
int dryValue = 600; // number in dry air (change me!)
int wetValue = 300; // number in water (change me!)

void setup() {
  pinMode(ledPin, OUTPUT);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.setTextColor(SSD1306_WHITE);
}

void loop() {
  int raw = analogRead(soilPin);
  int percent = map(raw, dryValue, wetValue, 0, 100);
  percent = constrain(percent, 0, 100); // keep 0 to 100

  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("PLANT MONITOR");
  display.setTextSize(2);
  display.setCursor(0, 16);
  display.print(percent);
  display.println("%");
  display.drawRect(0, 40, 128, 14, SSD1306_WHITE);
  display.fillRect(2, 42, percent * 124 / 100, 10, SSD1306_WHITE);

  if (percent < 30) {
    digitalWrite(ledPin, HIGH); // thirsty!
    display.setTextSize(1);
    display.setCursor(0, 56);
    display.print("WATER ME!");
  } else {
    digitalWrite(ledPin, LOW);
  }
  display.display();
  delay(500);
}
```

HOW IT WORKS

- Wet soil gives a smaller number with this sensor. So we map dryValue to 0% and wetValue to 100%.
- **constrain(x, 0, 100)** keeps the number inside the limits.
- The bar is a filled rectangle. Its width grows with the percent.

TRY IT YOURSELF
Add a buzzer that beeps once when the soil becomes too dry.

ROBO SAYS
Do not put the electronic part of the sensor in water. Only the long probe goes in the soil.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

18 Smart Parking Helper

Ultrasonic + LEDs + Buzzer | 40 min

WHAT IS IT?

Some cars beep faster as they get close to a wall. You will build that system! Green means safe, yellow means careful, red means stop. The buzzer beeps faster when the wall is near.

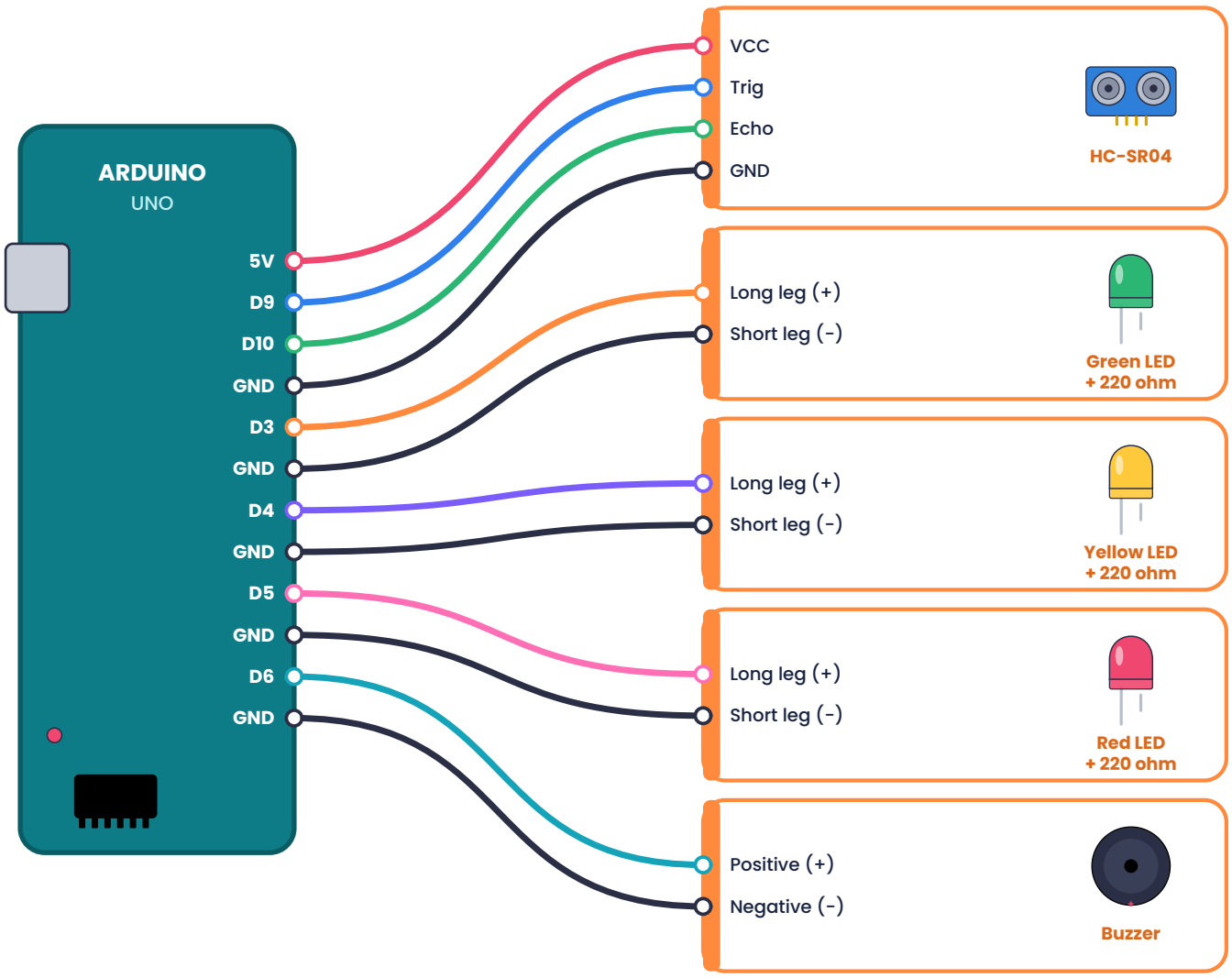
OBJECTIVE

Join a sensor with three LEDs and a buzzer, and use if / else if / else.

HARDWARE YOU NEED

Arduino UNO x1	HC-SR04 sensor x1	Green LED x1	Yellow LED x1
Red LED x1	Buzzer x1	Resistor 220 ohm x3	Breadboard x1

CONNECTION DIAGRAM



Use the breadboard GND rail so that all short legs and the buzzer share one GND wire.

STEP-BY-STEP PROCEDURE

1. Build the sensor circuit from Project 14.
2. Build three LED circuits on pins **D3** (green), **D4** (yellow) and **D5** (red).
3. Connect the buzzer to **D6** and GND.
4. Upload the code.
5. Move a book towards the sensor. Watch the lights change and listen to the beeps.

THE CODE (TYPE IT OR COPY IT)

```
const int trigPin = 9, echoPin = 10;
const int greenLed = 3, yellowLed = 4, redLed = 5;
const int buzzer = 6;

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(greenLed, OUTPUT);
  pinMode(yellowLed, OUTPUT);
  pinMode(redLed, OUTPUT);
  pinMode(buzzer, OUTPUT);
}

float getDistance() { // our own measuring function
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long t = pulseIn(echoPin, HIGH, 30000); // wait max 30 ms
  return t * 0.034 / 2;
}

void loop() {
  float d = getDistance();
  digitalWrite(greenLed, LOW);
  digitalWrite(yellowLed, LOW);
  digitalWrite(redLed, LOW);

  if (d > 30 || d == 0) { // far away: safe
    digitalWrite(greenLed, HIGH);
  } else if (d > 15) { // coming closer
    digitalWrite(yellowLed, HIGH);
    tone(buzzer, 1000, 80);
    delay(400);
  } else { // too close!
    digitalWrite(redLed, HIGH);
    tone(buzzer, 1500, 80);
    delay(100);
  }
  delay(60);
}
```

HOW IT WORKS

- **float getDistance()** is a function that gives back (returns) a number.
- **if / else if / else** checks the conditions from top to bottom.
- A short delay means fast beeps. A long delay means slow beeps.

TRY IT YOURSELF
Change the distances (30 and 15) to fit your table. Add a fourth zone with a faster beep.

ROBO SAYS
pulseIn(..., 30000) stops waiting after 30 ms. Without it the program can freeze when nothing is in front.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

SECTION 4 | LEVEL: INVENTOR

Advanced Projects

Motors, robots and smart systems.



Time to be an inventor! Here you build moving robots and complete smart machines. These projects join many parts together, so build slowly and test after every small step.

WHAT YOU WILL LEARN

- Drive motors with a motor driver
- Control speed and direction
- Build robots that follow lines and avoid walls
- Control a car with your phone
- Create complete systems with many sensors and outputs

PROJECTS IN THIS SECTION

No.	Project	New skill	Level	Time
19	**Motor Driver Basics**	DC Motor + L298N	★★★	40 min
20	**Speed and Direction Control**	Motor + Knob + Button	★★★	40 min
21	**Line Follower Robot**	Robot Car + IR Sensors	★★★	60 min
22	**Obstacle-Avoiding Robot**	Robot + Ultrasonic + Servo	★★★	70 min
23	**Bluetooth Remote Car**	Robot + Bluetooth	★★★	60 min
24	**Mini Weather Station**	Many Sensors + OLED	★★★	50 min
25	**Secret Door Lock**	Keypad + Servo + LCD + Buzzer	★★★	75 min

ROBO SAYS
Ask an adult to help with batteries. Always join all the grounds (GND) together, and unplug the power before you move a wire.

19 Motor Driver Basics

DC Motor + L298N | 40 min

★ ★ ★

WHAT IS IT?

A motor needs much more power than an Arduino pin can give. So we use a motor driver, a strong switch that the Arduino controls. You will spin a motor forward, stop it and spin it backward.

OBJECTIVE

Drive a DC motor in two directions using the L298N motor driver.

HARDWARE YOU NEED



Arduino UNO x1



L298N motor driver x1



DC motor (gear motor) x1

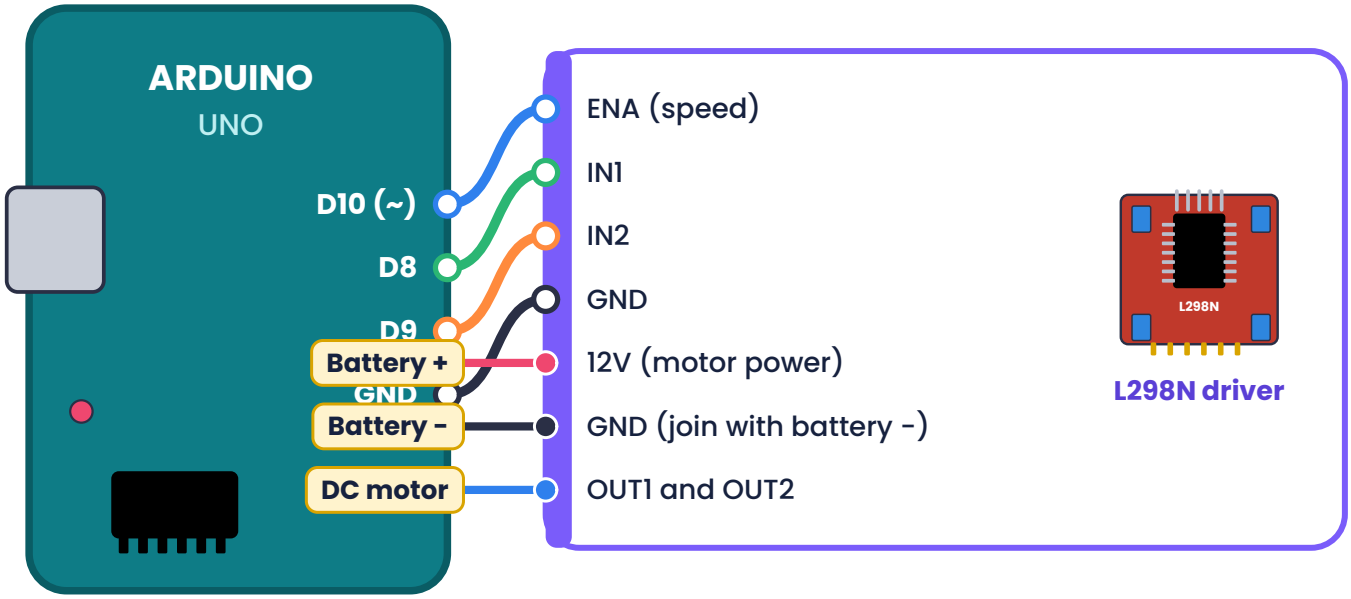


Battery pack 6-9 V x1



Jumper wires x6

CONNECTION DIAGRAM



Remove the small jumper cap on ENA, so that the Arduino can control the speed. Join battery minus with Arduino GND.

CAREFUL!

Do not touch the L298N chip. It can become hot. Always join the grounds of the battery and the Arduino.

STEP-BY-STEP PROCEDURE

1. Power OFF. Connect the motor wires to the screw terminals **OUT1** and **OUT2**.
2. Connect the battery red wire to **12V** and the black wire to **GND** on the driver.
3. Join the driver GND to the Arduino **GND**. All grounds must be joined!
4. Take the jumper cap off ENA. Join ENA to **D10**, IN1 to **D8**, IN2 to **D9**.
5. Upload the code with USB. Then connect the battery.
6. The motor goes forward, stops, goes backward and stops. If the direction is wrong, swap the two motor wires.

THE CODE (TYPE IT OR COPY IT)

```
const int ENA = 10; // speed pin (PWM)
const int IN1 = 8;
const int IN2 = 9;

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
}

// our own function: a and b choose the direction
void motor(int a, int b, int speedValue) {
  digitalWrite(IN1, a);
  digitalWrite(IN2, b);
  analogWrite(ENA, speedValue); // speed 0 to 255
}

void loop() {
  motor(HIGH, LOW, 200); // FORWARD
  delay(2000);
  motor(LOW, LOW, 0); // STOP
  delay(1000);
  motor(LOW, HIGH, 200); // BACKWARD
  delay(2000);
  motor(LOW, LOW, 0); // STOP
  delay(1000);
}
```

HOW IT WORKS

- motor(HIGH, LOW, 200) = one direction. motor(LOW, HIGH, 200) = the other direction.
- The speed number goes to ENA with PWM. A big number means a fast motor.
- The motor takes power from the battery. The Arduino only sends small signals.

TRY IT YOURSELF
Make the motor start slowly, speed up to full speed and then slow down again.

ROBO SAYS
Never connect a motor straight to an Arduino pin. It can damage the board.

20 Speed and Direction Control

Motor + Knob + Button | 40 min

★ ★ ★

WHAT IS IT?

Now you are the driver! Turn a knob to change the motor speed. Press a button to change the direction. This is how a remote-control car works.

OBJECTIVE

Control a motor using two inputs: a potentiometer for speed and a button for direction.

HARDWARE YOU NEED



Arduino UNO x1



L298N motor driver x1



DC motor x1



Potentiometer 10k x1

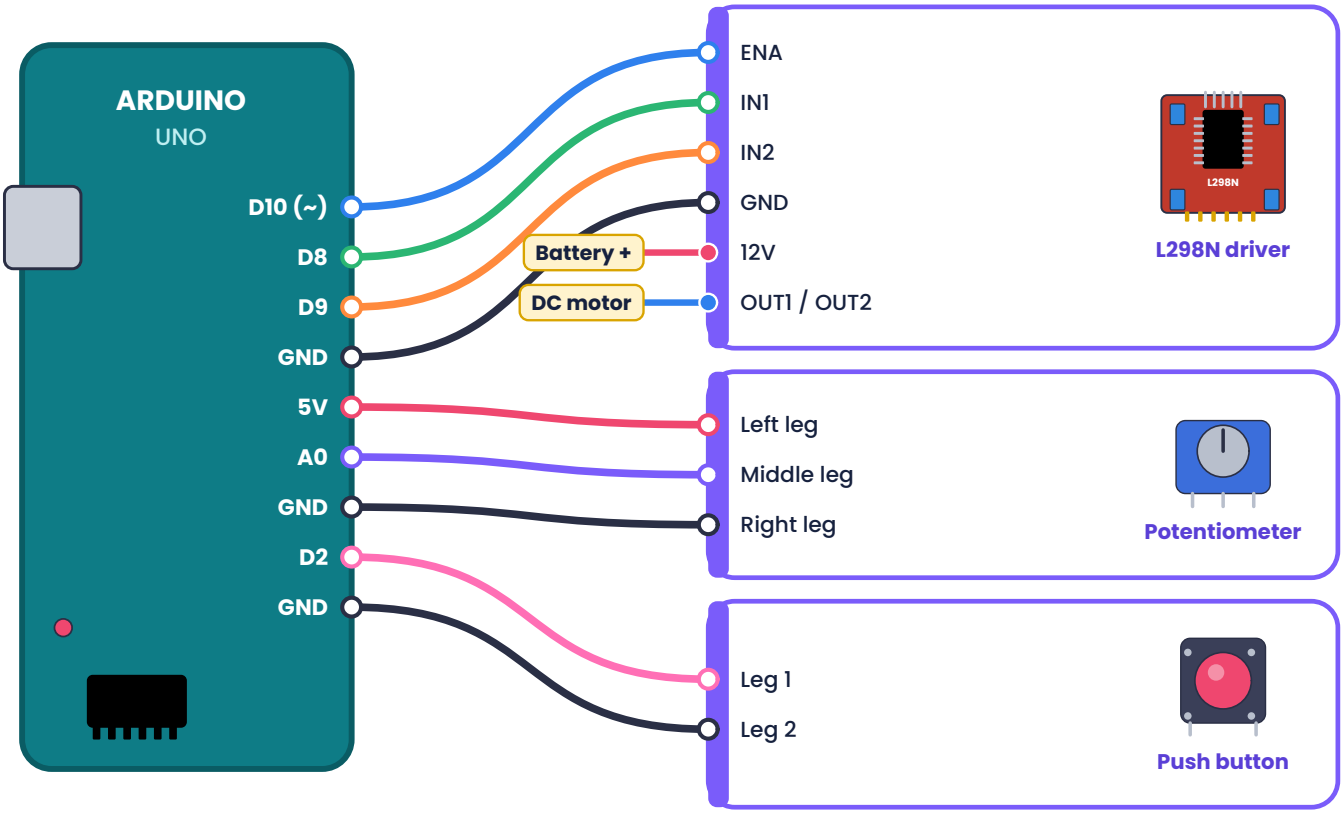


Push button x1



Battery pack 6-9 V x1

CONNECTION DIAGRAM



ⓘ Build the motor circuit from Project 19 first. Then add the knob and the button.

STEP-BY-STEP PROCEDURE

1. Build the motor driver circuit from Project 19.
2. Add the potentiometer: left leg to 5V, middle leg to **A0**, right leg to GND.
3. Add the push button between **D2** and GND.
4. Upload the code, then connect the battery.
5. Turn the knob to change the speed. Press the button to turn the motor around.

THE CODE (TYPE IT OR COPY IT)

```
const int ENA = 10, IN1 = 8, IN2 = 9;
const int knobPin = A0;
const int buttonPin = 2;

bool forward = true; // direction memory
bool lastButton = HIGH;

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
}

void loop() {
  // 1. read the button, change direction on each press
  bool button = digitalRead(buttonPin);
  if (lastButton == HIGH && button == LOW) {
    forward = !forward; // flip the direction
    delay(50); // small wait (debounce)
  }
  lastButton = button;

  // 2. read the knob, make speed
  int speedValue = map(analogRead(knobPin), 0, 1023, 0, 255);

  // 3. drive the motor
  digitalWrite(IN1, forward ? HIGH : LOW);
  digitalWrite(IN2, forward ? LOW : HIGH);
  analogWrite(ENA, speedValue);
}
```

HOW IT WORKS

- **bool** is a variable that is only true or false. **forward = !forward** flips it.
- We save the old button state (**lastButton**) so that one press counts only once.
- **forward ? HIGH : LOW** is a short way to write if / else.

TRY IT YOURSELF
Add two LEDs: a green one for forward and a red one for backward.

ROBO SAYS
The motor may not move at very low speed numbers. That is normal. It needs some power to start.

21 Line Follower Robot

Robot Car + IR Sensors | 60 min

★ ★ ★

WHAT IS IT?

Your first real robot! It follows a black line on the floor all by itself. Two infrared (IR) sensors look at the floor. The Arduino reads them and steers the two motors.

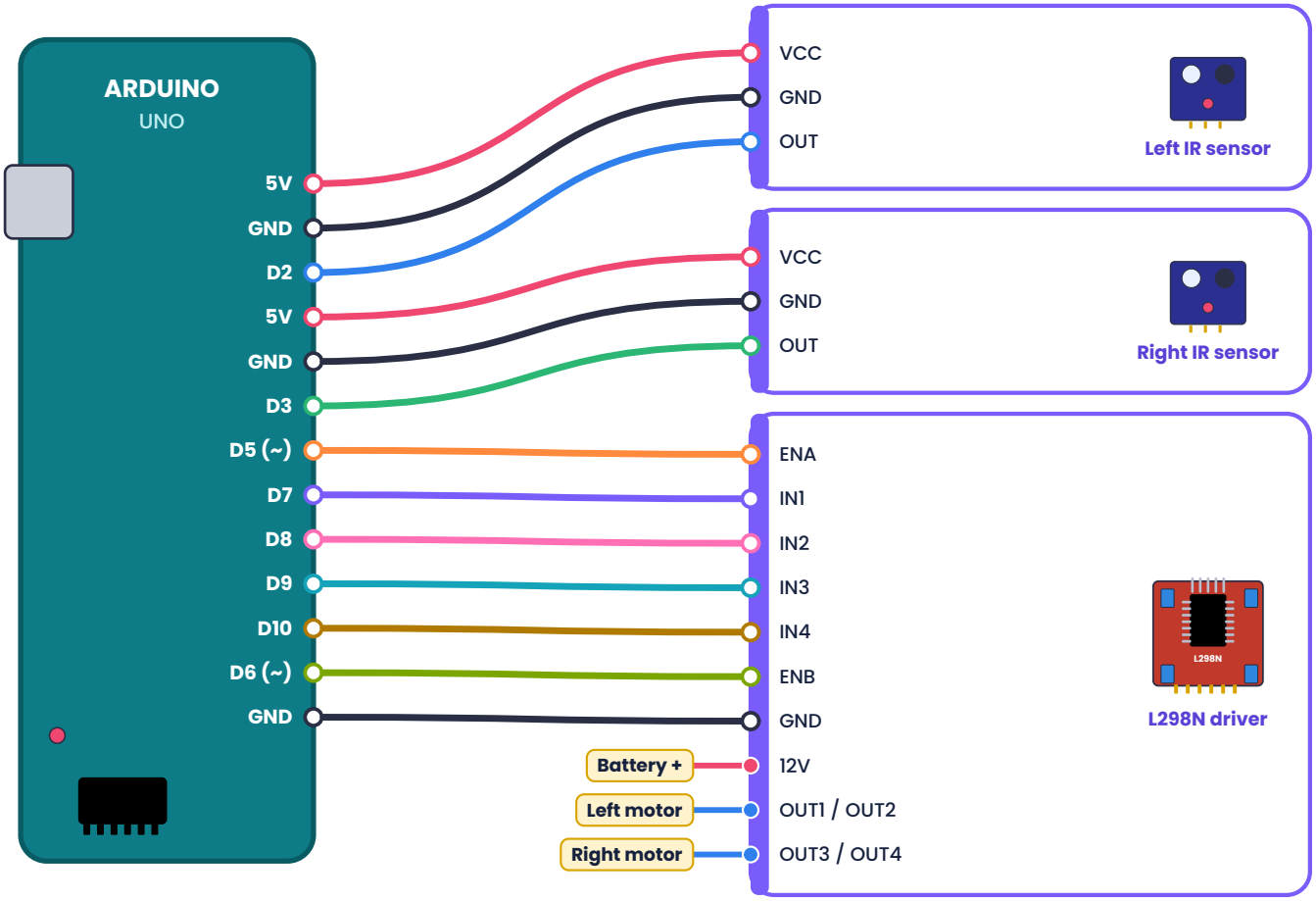
OBJECTIVE

Build a 2-wheel robot that reads two IR sensors and follows a line.

HARDWARE YOU NEED

- Arduino UNO — x1
- L298N motor driver — x1
- Gear motors + wheels — x2
- IR line sensor — x2
- Battery pack 7-9 V — x1
- Robot chassis + caster — x1

CONNECTION DIAGRAM



Fix the two IR sensors at the front of the robot, a little above the floor (about 1 cm), with a gap a bit wider than the line.

CAREFUL!

Keep fingers away from the wheels. Put the robot on a book to test the wheels first.

STEP-BY-STEP PROCEDURE

1. Build the chassis. Fix the two motors, the wheels and the caster wheel.
2. Fix the L298N and the Arduino on the chassis. Connect the motors: left to OUT1/OUT2 and right to OUT3/OUT4.
3. Remove the jumper caps on ENA and ENB. Connect the 6 control pins as in the diagram.
4. Mount the two IR sensors at the front. Connect VCC, GND and OUT.
5. Make a track with black tape (2 cm wide) on a white floor or paper.
6. Upload the code with USB. Then connect the battery and put the robot on the line.
7. If the robot turns the wrong way, swap the values of **LINE** and **FLOOR** in the code.

THE CODE (TYPE IT OR COPY IT)

```
const int ENA = 5, IN1 = 7, IN2 = 8; // left motor
const int IN3 = 9, IN4 = 10, ENB = 6; // right motor
const int leftIR = 2, rightIR = 3;

const int LINE = HIGH; // value when the sensor sees the black line
int carSpeed = 150;

void setup() {
  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(leftIR, INPUT);
  pinMode(rightIR, INPUT);
}

void drive(int l1, int l2, int r1, int r2) {
  digitalWrite(IN1, l1); digitalWrite(IN2, l2);
  digitalWrite(IN3, r1); digitalWrite(IN4, r2);
  analogWrite(ENA, carSpeed);
  analogWrite(ENB, carSpeed);
}
void goForward() { drive(HIGH, LOW, HIGH, LOW); }
void turnLeft() { drive(LOW, LOW, HIGH, LOW); } // only the right wheel moves
void turnRight() { drive(HIGH, LOW, LOW, LOW); } // only the left wheel moves
void stopCar() { drive(LOW, LOW, LOW, LOW); }

void loop() {
  bool left = (digitalRead(leftIR) == LINE); // true = on the line
  bool right = (digitalRead(rightIR) == LINE);

  if (!left && !right) {
    goForward(); // line is in the middle
  } else if (left && !right) {
    turnLeft(); // line moved to the left
  } else if (!left && right) {
    turnRight(); // line moved to the right
  } else {
    stopCar(); // both on black: stop
  }
}
```

HOW IT WORKS

- Black paper soaks up IR light. White paper reflects it. So the two sensors give different signals.
- If the LEFT sensor sees black, the line is going left. So the robot turns left to follow it.
- We wrote small functions: goForward(), turnLeft(), turnRight(), stopCar(). The loop() stays short and clear.

TRY IT YOURSELF
Make the robot slow down at turns. Hint: change carSpeed inside turnLeft() and turnRight().

ROBO SAYS
Test the sensors first. Each IR module has a small LED that lights up when it sees something. Turn the blue screw to tune it.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

22 Obstacle-Avoiding Robot

Robot + Ultrasonic + Servo | 70 min

★ ★ ★

WHAT IS IT?

This robot has eyes! An ultrasonic sensor on a servo looks forward. When a wall is close, the robot stops, looks left and right, and drives to the side with more space.

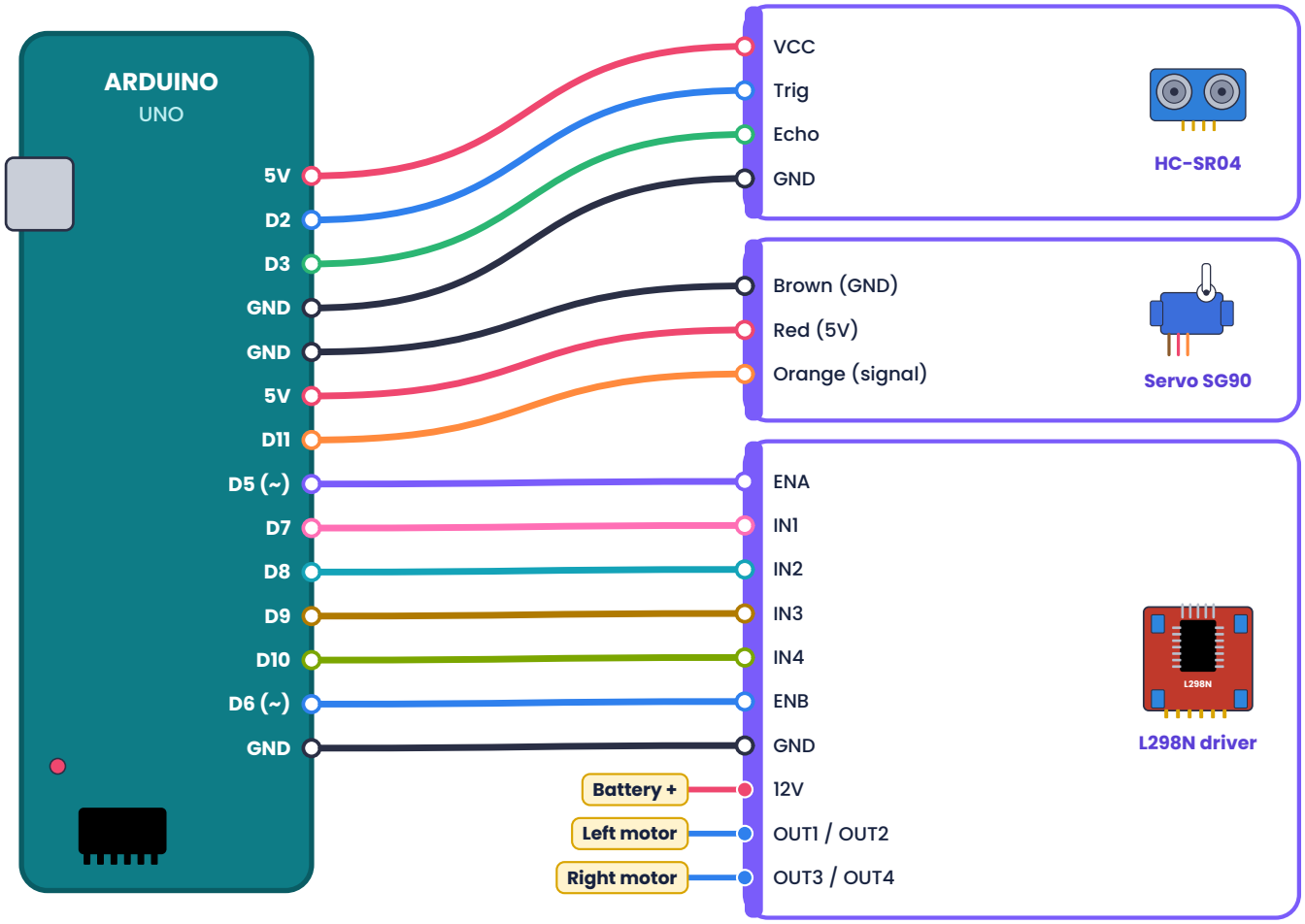
OBJECTIVE

Join a distance sensor, a servo and two motors in one program that makes decisions.

HARDWARE YOU NEED

- Arduino UNO — x1
- L298N motor driver — x1
- Gear motors + wheels — x2
- HC-SR04 sensor — x1
- Servo SG90 — x1
- Battery pack 7-9 V — x1

CONNECTION DIAGRAM



Glue or tape the ultrasonic sensor on the servo arm so it can look left and right. Servo 90 degrees = straight ahead.

> **CAREFUL!**
> The Servo library switches off analogWrite on pins 9 and 10. We only use them as ON/OFF pins, so it is safe here.

STEP-BY-STEP PROCEDURE

1. Build the robot car like in Project 21 (chassis, motors, L298N, battery).
2. Fix the servo at the front. Tape the ultrasonic sensor on top of the servo arm.
3. Connect the sensor, the servo and the motor driver as in the diagram.
4. Upload the code with USB. Then connect the battery.
5. Put the robot on the floor. Place a box in front of it and watch it choose a new way.
6. If a wheel spins the wrong way, swap its two motor wires.

THE CODE (TYPE IT OR COPY IT)

```
const int ENA = 5, IN1 = 7, IN2 = 8; // left motor
const int IN3 = 9, IN4 = 10, ENB = 6; // right motor
int carSpeed = 150;

void drive(int l1, int l2, int r1, int r2) {
  digitalWrite(IN1, l1); digitalWrite(IN2, l2);
  digitalWrite(IN3, r1); digitalWrite(IN4, r2);
  analogWrite(ENA, carSpeed);
  analogWrite(ENB, carSpeed);
}
void goForward() { drive(HIGH, LOW, HIGH, LOW); }
void goBackward() { drive(LOW, HIGH, LOW, HIGH); }
void turnLeft() { drive(LOW, HIGH, HIGH, LOW); } // spin on the spot
void turnRight() { drive(HIGH, LOW, LOW, HIGH); }
void stopCar() { drive(LOW, LOW, LOW, LOW); }

#include <Servo.h>
const int trigPin = 2, echoPin = 3;
Servo head;

int readDistance() {
  digitalWrite(trigPin, LOW); delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long t = pulseIn(echoPin, HIGH, 30000);
  if (t == 0) return 200; // no echo = nothing close
  return t * 0.034 / 2;
}

void setup() {
  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  head.attach(11);
  head.write(90); // look straight ahead
  delay(500);
}

void loop() {
```

```
  int front = readDistance();
  if (front > 25) {
    goForward(); // road is free
  } else {
    stopCar(); delay(200);
    goBackward(); delay(300);
    stopCar();

    head.write(170); delay(500); // look left
    int leftSpace = readDistance();
    head.write(10); delay(500); // look right
    int rightSpace = readDistance();
    head.write(90); delay(300); // look ahead again

    if (leftSpace > rightSpace) turnLeft(); else turnRight();
    delay(450);
    stopCar();
  }
  delay(30);
}
```

HOW IT WORKS

- The robot checks the front distance again and again. This is a **sense - think - act** loop.
- When the way is blocked it compares leftSpace and rightSpace, then turns to the bigger one.
- Change **delay(450)** to make the robot turn more or less.

TRY IT YOURSELF
Add a buzzer that beeps once each time the robot sees an obstacle.

ROBO SAYS
Use fresh batteries. When the battery is weak, the motors become slow and the robot may restart.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

23 Bluetooth Remote Car

Robot + Bluetooth | 60 min

★ ★ ★

WHAT IS IT?

Control your robot with your phone! The HC-05 Bluetooth module receives letters from the phone. F means forward, B backward, L left, R right and S stop.

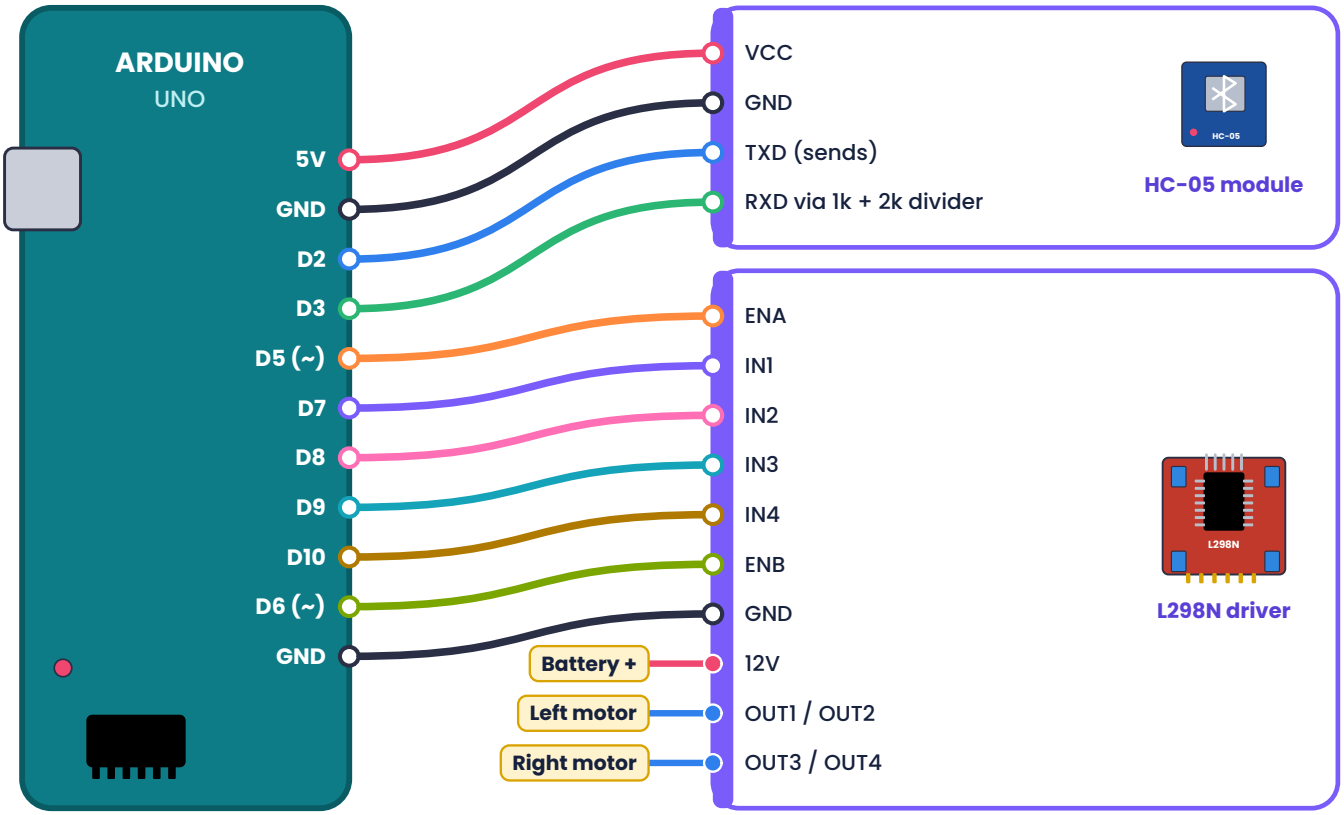
OBJECTIVE

Receive commands from a phone with Bluetooth and turn them into robot movement.

HARDWARE YOU NEED

- Arduino UNO — x1
- L298N motor driver — x1
- Gear motors + wheels — x2
- HC-05 Bluetooth — x1
- Battery pack 7-9 V — x1
- Resistors 1k and 2k — x2

CONNECTION DIAGRAM



HC-05 RXD only likes 3.3 V. Use a divider: Arduino D3 - 1k resistor - RXD, and RXD - 2k resistor - GND.

CAREFUL!
Power off the robot before you change any wire. Always check the divider on RXD.

STEP-BY-STEP PROCEDURE

1. Build the robot car like in Project 21 or 22.
2. Connect the HC-05: VCC to 5V, GND to GND, TXD to **D2**.
3. Build the voltage divider for RXD: D3 to a 1k resistor to RXD. Then a 2k resistor from RXD to GND.
4. Upload the code with USB. Then connect the battery.
5. On your phone, install a free app like **Serial Bluetooth Terminal**. Pair with HC-05 (password 1234 or 0000).
6. Open the app, connect, and send letters: F, B, L, R, S. The robot moves!

THE CODE (TYPE IT OR COPY IT)

```
const int ENA = 5, IN1 = 7, IN2 = 8; // left motor
const int IN3 = 9, IN4 = 10, ENB = 6; // right motor
int carSpeed = 150;

void drive(int l1, int l2, int r1, int r2) {
  digitalWrite(IN1, l1); digitalWrite(IN2, l2);
  digitalWrite(IN3, r1); digitalWrite(IN4, r2);
  analogWrite(ENA, carSpeed);
  analogWrite(ENB, carSpeed);
}
void goForward() { drive(HIGH, LOW, HIGH, LOW); }
void goBackward() { drive(LOW, HIGH, LOW, HIGH); }
void turnLeft() { drive(LOW, HIGH, HIGH, LOW); } // spin on the spot
void turnRight() { drive(HIGH, LOW, LOW, HIGH); }
void stopCar() { drive(LOW, LOW, LOW, LOW); }

#include <SoftwareSerial.h>
SoftwareSerial bt(2, 3); // RX = D2, TX = D3

void setup() {
  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  bt.begin(9600); // HC-05 normal speed
  stopCar();
}

void loop() {
  if (bt.available()) { // did a letter arrive?
    char cmd = bt.read();
    switch (cmd) {
      case 'F': goForward(); break;
      case 'B': goBackward(); break;
      case 'L': turnLeft(); break;
      case 'R': turnRight(); break;
      case 'S': stopCar(); break;
    }
  }
}
```

HOW IT WORKS

- Bluetooth sends letters (characters) one by one. **bt.read()** picks up each letter.
- **SoftwareSerial** makes extra talking pins, so USB stays free for uploading.
- **switch / case** is a clean way to choose between many options.

TRY IT YOURSELF
Add the letters 'W' and 'X' to make the robot faster and slower by changing carSpeed.

ROBO SAYS
If the module does not appear on your phone, check that its red LED is blinking. That means it is ready.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

24 Mini Weather Station

Many Sensors + OLED | 50 min

★ ★ ★

WHAT IS IT?

Put together everything you know! A temperature sensor, a light sensor and a screen make a real weather station. If it becomes too hot, the buzzer warns you.

OBJECTIVE

Combine three peripherals (DHT11, LDR and OLED) plus a buzzer alarm in one clean program.

HARDWARE YOU NEED



Arduino UNO x1



DHT11 sensor x1



LDR sensor x1



Resistor 10k ohm x1

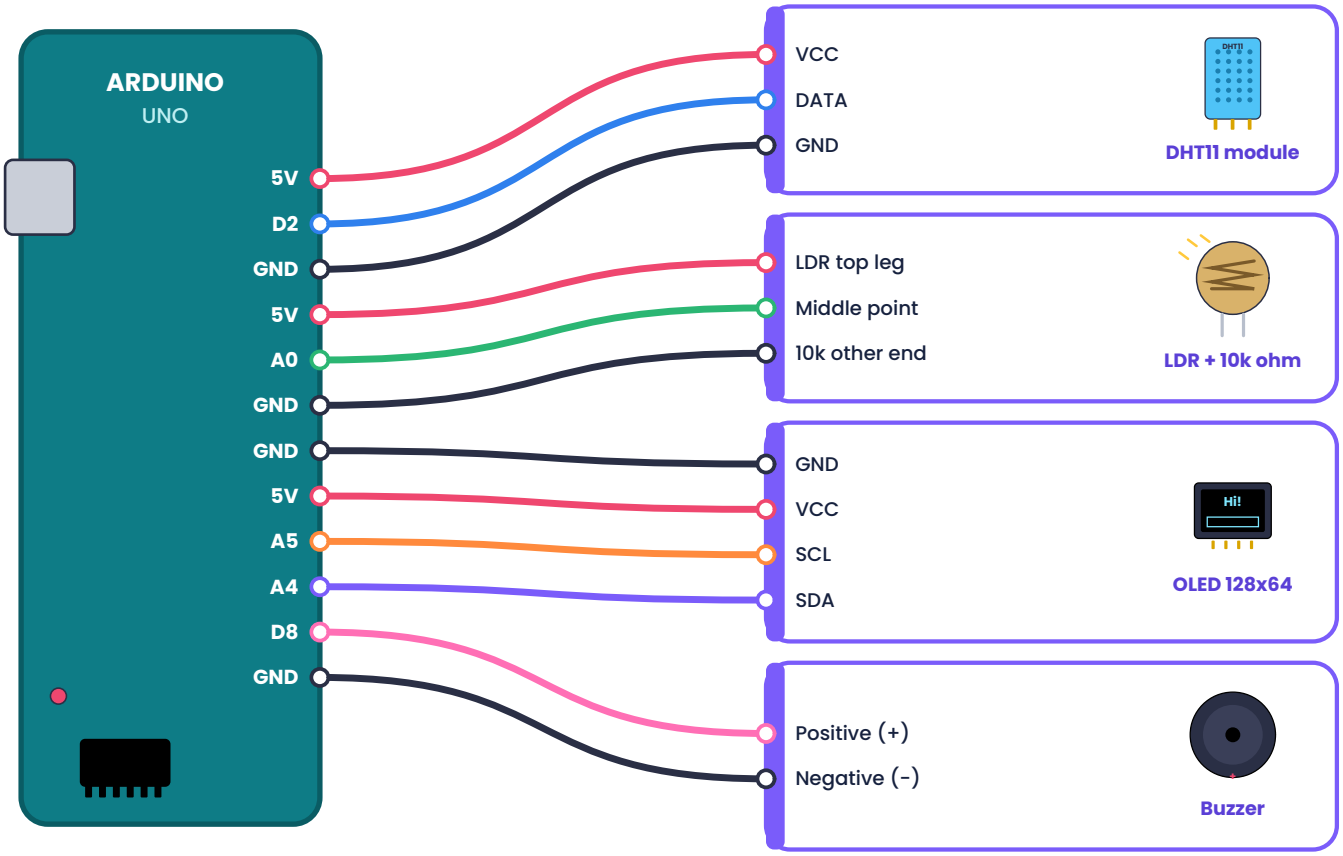


OLED I2C x1



Buzzer x1

CONNECTION DIAGRAM



Use the breadboard rails to share 5V and GND between all parts. Keep the wires short and tidy.

STEP-BY-STEP PROCEDURE

1. Connect the DHT11 to D2, the LDR divider to A0, the OLED to A4/A5 and the buzzer to D8.
2. Install the libraries from Projects 11 and 15 if you have not already.
3. Upload the code.
4. The screen shows temperature, humidity and light.
5. Hold your fingers around the DHT11. When the temperature goes above 35 C the buzzer beeps.

THE CODE (TYPE IT OR COPY IT)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

DHT dht(2, DHT11);
Adafruit_SSD1306 display(128, 64, &Wire, -1);
const int ldrPin = A0;
const int buzzer = 8;
const float hotLimit = 35.0; // alarm temperature in C

void setup() {
  pinMode(buzzer, OUTPUT);
  dht.begin();
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.setTextColor(SSD1306_WHITE);
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  int light = map(analogRead(ldrPin), 0, 1023, 0, 100); // percent

  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("MINI WEATHER STATION");
  display.drawLine(0, 10, 127, 10, SSD1306_WHITE);

  if (isnan(t) || isnan(h)) {
    display.setCursor(0, 24);
    display.println("Sensor error");
  } else {
    display.setCursor(0, 16);
    display.print("Temp : "); display.print(t, 1); display.println(" C");
    display.setCursor(0, 30);
    display.print("Humid: "); display.print(h, 0); display.println(" %");
    display.setCursor(0, 44);
    display.print("Light: "); display.print(light); display.println(" %");
    if (t > hotLimit) {
      display.setCursor(0, 56);
      display.print("!! TOO HOT !!");
      tone(buzzer, 2000, 300);
    }
  }
  display.display();
  delay(1500);
}
```

HOW IT WORKS

- Each part is easy alone. The skill is joining them. Add one part at a time and test.
- **hotLimit** is a constant at the top. It is easy to find and change.
- The program reads three sensors, thinks once, then shows everything on one screen.

TRY IT YOURSELF
Add a LED that turns on when it gets dark (light below 20 %). Add a bar for the humidity.

ROBO SAYS
Build step by step: first the OLED, then the DHT11, then the LDR, then the buzzer. Test after each step.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

25 Secret Door Lock

Keypad + Servo + LCD + Buzzer | 75 min

★ ★ ★

WHAT IS IT?

Build a safe with a secret PIN code! Type the right numbers on the keypad and the servo opens the door. Type a wrong code and the buzzer shouts. This is the biggest project in the booklet.

OBJECTIVE

Use a keypad, an LCD, a servo and a buzzer together to make a complete smart lock.

HARDWARE YOU NEED



Arduino UNO x1



4x4 keypad x1



16x2 LCD with I2C x1



Servo SG90 x1

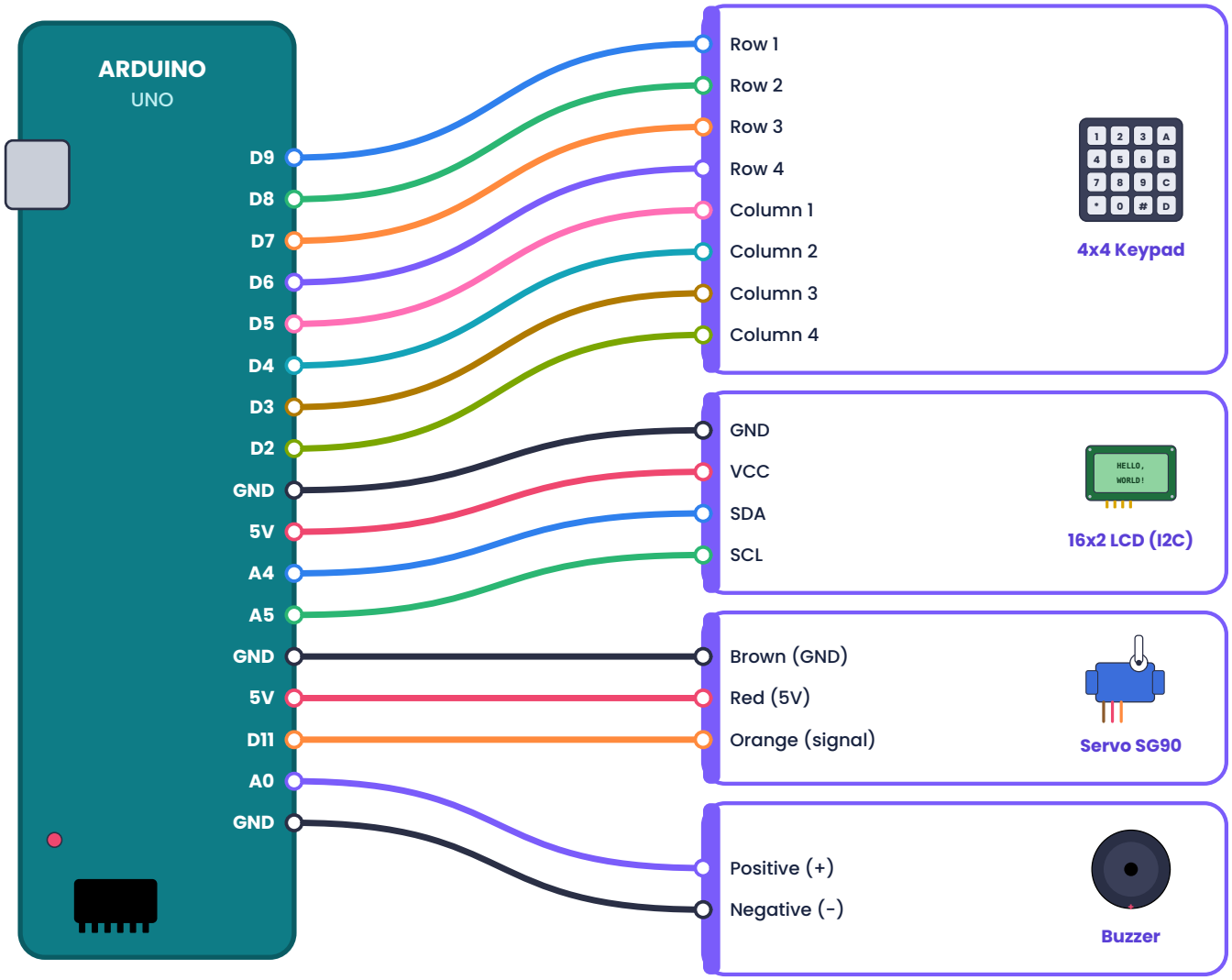


Buzzer x1



Jumper wires x16

CONNECTION DIAGRAM



The keypad has 8 pins in a row. Row pins come first, then column pins. Test each wire carefully.

STEP-BY-STEP PROCEDURE

1. Connect the 8 keypad pins to D9, D8, D7, D6, D5, D4, D3, D2 (in this order).
2. Connect the LCD (A4, A5), the servo (D11) and the buzzer (A0).
3. Install the library **Keypad** by Mark Stanley and Alexander Brevig.
4. Upload the code. The LCD says "Enter PIN".
5. Type **1234** and press **#**. The door opens for 4 seconds, then locks again.
6. Press * to clear what you typed. Change the secret PIN in the code!

THE CODE (TYPE IT OR COPY IT)

```
#include <Keypad.h>
#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {5, 4, 3, 2};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo lockServo;
const int buzzer = A0;

String secret = "1234"; // the secret PIN
String typed = ""; // what the user typed

void showPrompt() {
  lcd.clear();
  lcd.print("Enter PIN:");
  lcd.setCursor(0, 1);
}

void setup() {
  lcd.init();
  lcd.backlight();
  lockServo.attach(11);
  lockServo.write(0); // door locked
  pinMode(buzzer, OUTPUT);
  showPrompt();
}

void loop() {
  char key = keypad.getKey();
  if (!key) return; // no key pressed
  tone(buzzer, 1000, 50); // key click

  if (key == '#') { // OK button
    if (typed == secret) {
      lcd.clear(); lcd.print("Door OPEN :)");
      lockServo.write(90); // open
```

```
      delay(4000);
      lockServo.write(0); // lock again
    } else {
      lcd.clear(); lcd.print("Wrong PIN!");
      tone(buzzer, 200, 800);
      delay(1500);
    }
    typed = "";
    showPrompt();
  } else if (key == '*') { // clear button
    typed = "";
    showPrompt();
  } else if (typed.length() < 8) { // add a digit
    typed += key;
    lcd.print('*'); // hide the number
  }
}
```

HOW IT WORKS

- The keypad library scans rows and columns and tells us which key is pressed.
- **typed += key** adds a new character to the end of the text (String).
- The robot-style plan: **input** (keypad), **brain** (compare with secret), **output** (servo, LCD, buzzer).

TRY IT YOURSELF
Lock the door for 10 seconds after 3 wrong tries. Add a red LED and a green LED.

ROBO SAYS
Keep your secret PIN secret! Use letters A-D too. A longer PIN is harder to guess.

MY LAB NOTES

☐ I built it and it works. ☐ I finished the challenge. ☐ I can explain it to a friend.

What I learned, or what I want to try next:

APPENDIX A

Pin Cheat Sheet

Here you can see which Arduino pins each project uses. Power pins (5V and GND) are used by almost every project, so they are not listed.

No.	Project	Pins used
01	Hello World!	USB cable only
02	LED Blink	D13
03	Traffic Light	D10, D9, D8
04	Button Controls LED	D2, D8
05	Buzzer Melody	D8
06	Knob Reader	A0
07	Fading LED	D9
08	Night Light (LDR)	A0, D9
09	RGB Colour Mixer	D9, D10, D11
10	LCD Message Board	A4, A5
11	OLED Display	A5, A4
12	Servo Sweep	D9
13	Servo Knob Control	A0, D9
14	Distance Meter	D9, D10
15	Weather on the LCD	D2, A4, A5
16	Motion Alarm	D2, D8, D13
17	Plant Water Monitor	A0, A5, A4, D8
18	Smart Parking Helper	D9, D10, D3, D4, D5, D6
19	Motor Driver Basics	D10, D8, D9
20	Speed and Direction Control	D10, D8, D9, A0, D2
21	Line Follower Robot	D2, D3, D5, D7, D8, D9, D10, D6
22	Obstacle-Avoiding Robot	D2, D3, D11, D5, D7, D8, D9, D10, D6
23	Bluetooth Remote Car	D2, D3, D5, D7, D8, D9, D10, D6
24	Mini Weather Station	D2, A0, A5, A4, D8
25	Secret Door Lock	D9, D8, D7, D6, D5, D4, D3, D2, A4, A5, D11, A0

APPENDIX B

When Something Goes Wrong

Problem	Why it happens	What to try
Nothing happens after upload	The board or the port is wrong.	Tools > Board > Arduino Uno. Tools > Port > choose the Arduino port. Try another USB cable.
"Upload failed" or port is grey	The computer cannot see the board.	Unplug and plug the USB again. Try another USB port. Close the Serial Monitor.
Red message with an error	A typing mistake in the code.	Read the line number. Check ; { } and capital letters.
The LED does not light	The LED is the wrong way, or a wire is loose.	Turn the LED around. Check that GND is connected.
Strange symbols in Serial Monitor	The speed does not match.	Set the Serial Monitor to 9600 baud.
The button seems backwards	INPUT_PULLUP makes pressed = LOW.	This is normal. Use if (digitalRead(pin) == LOW).
The LCD shows white boxes only	The contrast is too high, or the address is wrong.	Turn the blue screw on the back. Try address 0x3F instead of 0x27.
The servo shakes or the board restarts	Not enough power.	Use a battery pack for servos and motors. Join all GND wires.
The motor does not turn	No battery power, or ENA jumper problem.	Check the battery. Check ENA (pin D10 or D5). Join battery GND with Arduino GND.
The robot turns the wrong way	Motor wires or sensor values are swapped.	Swap the two wires of that motor. Or swap values in the code.
Library not found	The library is not installed.	Sketch > Include Library > Manage Libraries. Search the name and install.

APPENDIX C

Words to Know

Actuator	A part that does something: motor, LED, buzzer.
Analog	A smooth value, like 0 to 1023.
Baud	The speed of serial talking. We use 9600.
Breadboard	A board to build circuits without solder.
Circuit	A loop where electricity can flow.
Code / Sketch	The instructions you write for the Arduino.
Debug	Find and fix a problem.
Digital	ON or OFF, HIGH or LOW.
GND	Ground. The 0 volt return path.

I2C	A way to talk with only two wires (SDA, SCL).
IDE	The program where you write and upload code.
Library	A box of ready-made code to use.
Microcontroller	A tiny computer on one chip.
PWM	Fast ON / OFF to make smooth power.
Sensor	A part that reads the world.
Serial Monitor	A window to see Arduino messages.
Servo	A motor that turns to an exact angle.
Voltage	The push of electricity, in volts (V).

APPENDIX D

My Project Tracker

Tick the box when your project works. Colour the stars: 1 star = I built it, 2 stars = I did the challenge, 3 stars = I can teach a friend.

Done	No.	Project	Date	My stars
☐	01	Hello World!		☆ ☆ ☆
☐	02	LED Blink		☆ ☆ ☆
☐	03	Traffic Light		☆ ☆ ☆
☐	04	Button Controls LED		☆ ☆ ☆
☐	05	Buzzer Melody		☆ ☆ ☆
☐	06	Knob Reader		☆ ☆ ☆
☐	07	Fading LED		☆ ☆ ☆
☐	08	Night Light (LDR)		☆ ☆ ☆
☐	09	RGB Colour Mixer		☆ ☆ ☆
☐	10	LCD Message Board		☆ ☆ ☆
☐	11	OLED Display		☆ ☆ ☆
☐	12	Servo Sweep		☆ ☆ ☆
☐	13	Servo Knob Control		☆ ☆ ☆
☐	14	Distance Meter		☆ ☆ ☆
☐	15	Weather on the LCD		☆ ☆ ☆
☐	16	Motion Alarm		☆ ☆ ☆
☐	17	Plant Water Monitor		☆ ☆ ☆
☐	18	Smart Parking Helper		☆ ☆ ☆
☐	19	Motor Driver Basics		☆ ☆ ☆
☐	20	Speed and Direction Control		☆ ☆ ☆
☐	21	Line Follower Robot		☆ ☆ ☆
☐	22	Obstacle-Avoiding Robot		☆ ☆ ☆
☐	23	Bluetooth Remote Car		☆ ☆ ☆
☐	24	Mini Weather Station		☆ ☆ ☆
☐	25	Secret Door Lock		☆ ☆ ☆

CERTIFICATE

OF ACHIEVEMENT

This certificate is proudly given to

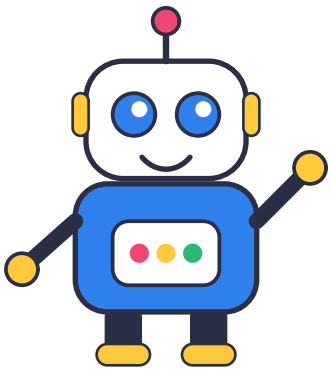
_____

(write your name here)

for completing all 25 projects in

Little Makers

Arduino and Robotics for Beginners



_____ Date

_____ Teacher or parent

Designed by Engr. Abdul Basit